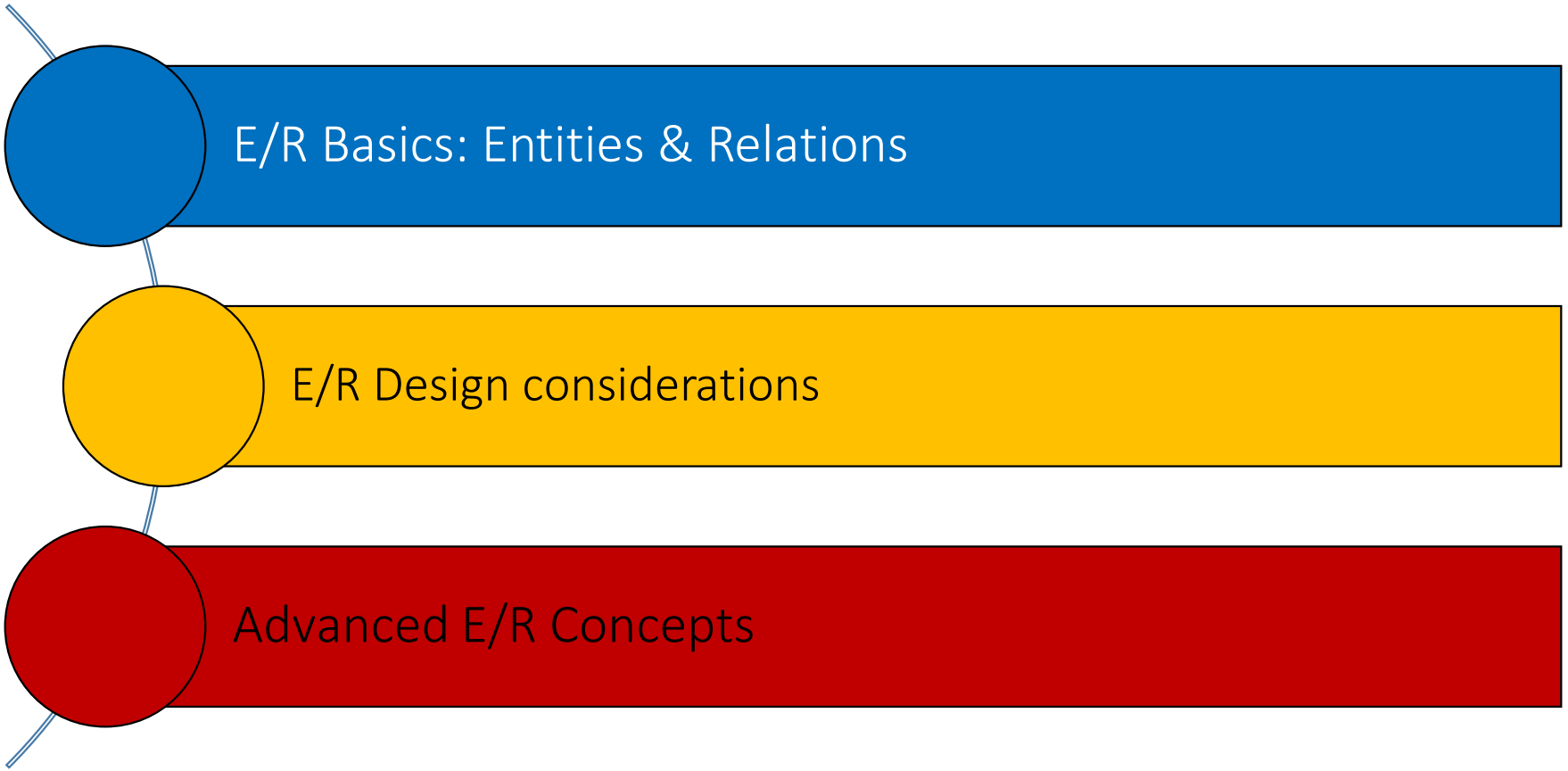
CSE 303: Database

Lecture 12

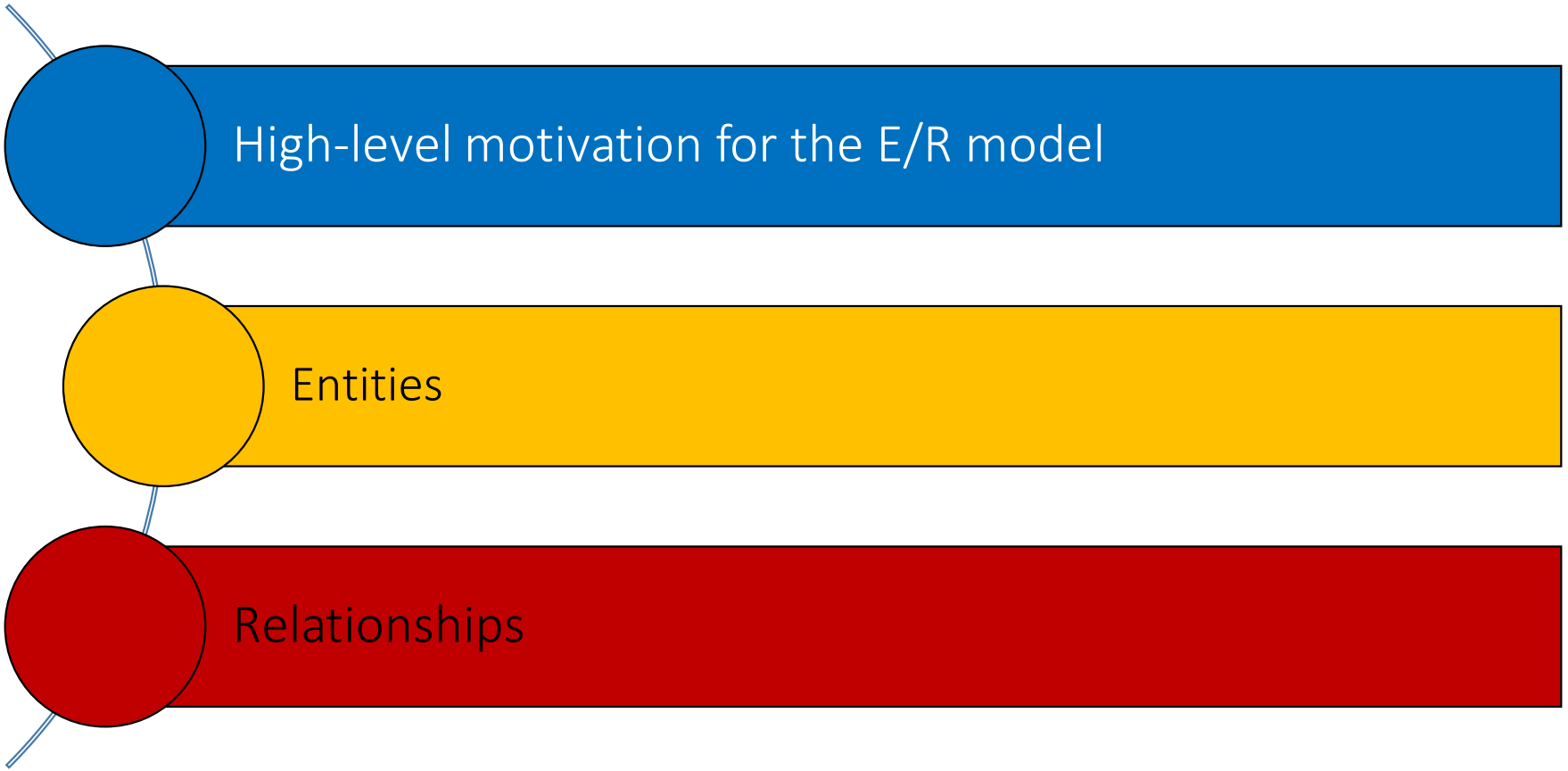
2022
Chapter 4: E-R diagram

Outline


E/R Basics: Entities & Relations
E/R Design considerations
Advanced E/R Concepts

1. E/R Basics: Entities & Relations

What you will learn about in this section



Database Design

- **Database design: Why do we need it?**
  - Agree on structure of the database before deciding on a particular implementation

- **Consider issues such as:**
  - What entities to model
  - How entities are related
  - What constraints exist in the domain
  - How to achieve good designs

- **Several formalisms exist**
  - We discuss one flavor of E/R diagrams

Database Design Process

1. Requirements Analysis | 2. Conceptual Design | 3. Logical, Physical, Security, etc.

1. Requirements analysis

- What is going to be stored?
- How is it going to be used?
- What are we going to do with the data?
- Who should access the data?

Technical and non-technical people are involved

Taught in course on system analysis and design

Database Design Process

1. Requirements Analysis | 2. Conceptual Design | 3. Logical, Physical, Security, etc.

2. Conceptual Design

- A high-level description of the database
- Sufficiently precise that technical people can understand it
- But, not so precise that non-technical people can't participate

This is where E/R diagram fits in.

Database Design Process

1. Requirements Analysis → 2. Conceptual Design → 3. Logical, Physical, Security, etc.

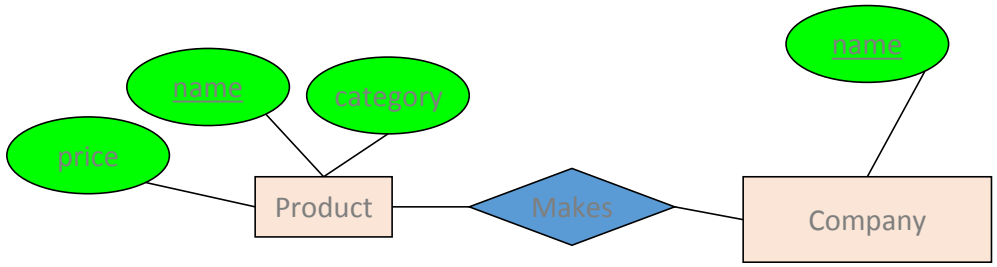
3. More:

- Logical Database Design
  - Translation of ER diagram to a relational database schema (description of tables)
- Physical Database Design
- Security Design

Database Design Process

1. Requirements Analysis → 2. Conceptual Design → 3. Logical, Physical, Security, etc.

E/R Model & Diagrams used



This process is iterated **many** times

E/R diagram is a *visual syntax* for DB design which is ***precise enough*** for technical points, but ***abstracted enough*** for non-technical people

Requirement Engineering

- **Tasks** and **techniques** that lead to an **understanding** of **requirements** is called **requirement engineering**.
- Requirement engineering **provides** appropriate m**echanism** for **understanding**

  - What customer wants
  - Analyzing needs
  - Assessing feasibility
  - Negotiating a reasonable solution
  - Specifying solution unambiguously
  - Validating the specification
  - Managing requirements

Requirements Engineering is Hard

"The **hardest** single **part** of building a software system is **deciding what to build**. No part of the work so cripples the resulting system if done wrong."

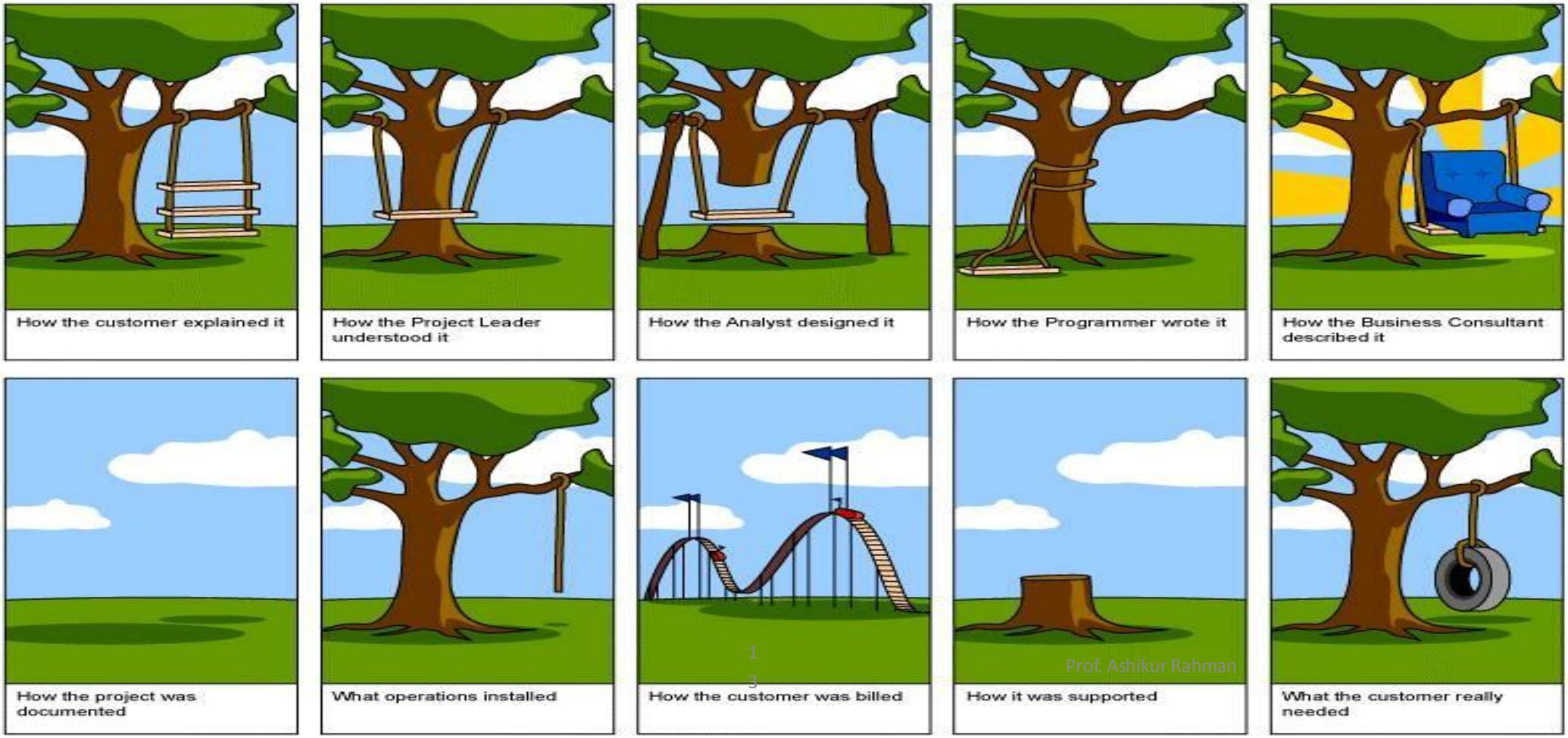
"Fred" Brooks Jr. is an American computer architect, software engineer, and computer scientist, best known for managing the development of IBM's System/360 family of computers

"The **seeds** of major software **disasters** are usually **shown** in the **first three months** of commencing the software project."

Capers Jones is an American specialist in software engineering methodologies

What is the right system to build ?

What is the right system to build ?

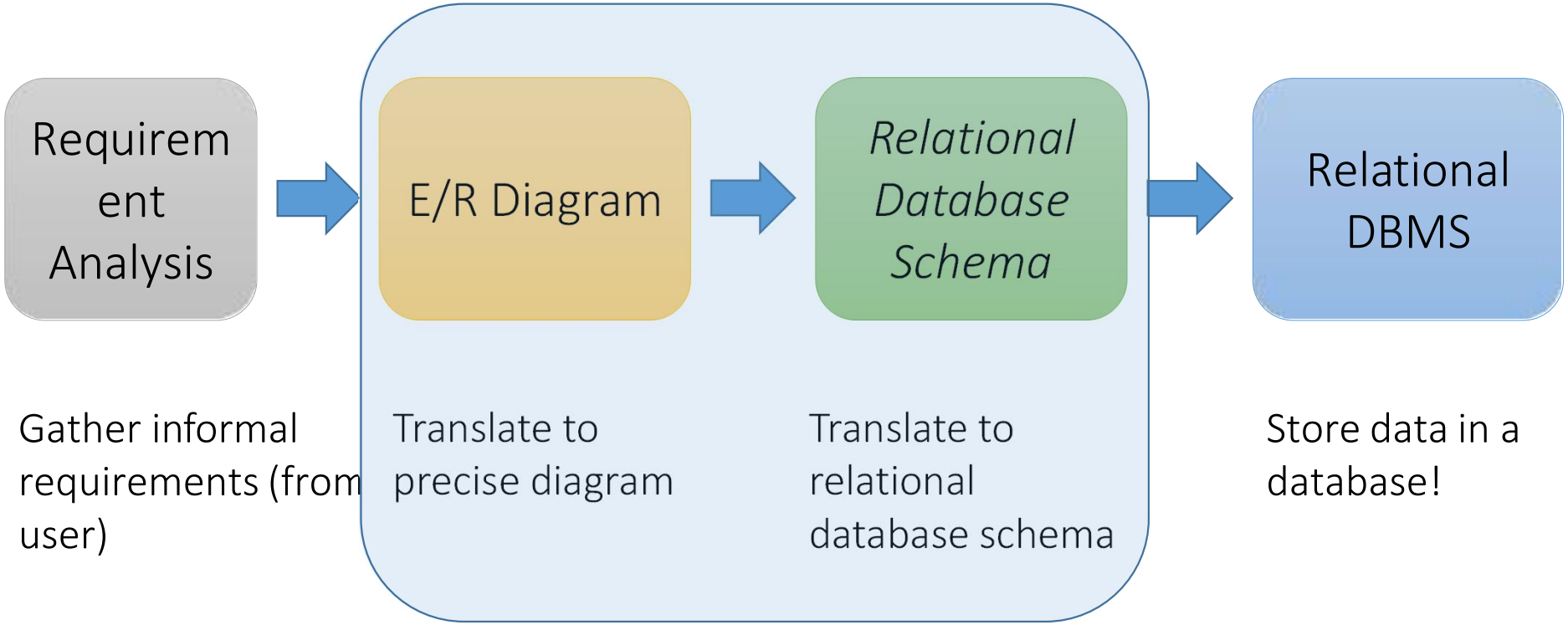


Requirement Analysis: Example

- For actors and directors, we want to store their name, a unique identification number, address and birthday (why not age?)
- For actors, we also want to store a photograph
- For films, we want to store the title, year of production and type (thriller, comedy, etc.)
- We want to know who directed and who acted in each film. Every film has one director. We store the salary of each actor for each film
- Etc…

Next Step: E/R modeling

- **Goal** of modeling is to translate informal requirements to a precise diagram. This diagram can then be translated into the desired **data model**, to allow data to be stored in a database



Interlude: Impact of the ER model

- The E/R model is one of the most cited articles in Computer Science
  - *"The Entity-Relationship model – toward a unified view of data"* Peter Chen, 1976

https://citeseerx.ist.psu.edu › viewdoc › download PDF

The Entity-Relationship Model-Toward a Unified ... - CiteSeerX

by PPINS CHEN · 197 · Cited by 12756 – A **data** model, called **the entity-relationship model**, is proposed. This model incorporates some of the important semantic information...

- Used by companies big and small
  - You'll know it soon enough

Basic Concepts: Entities, Attributes, Relationships

Entities and Entity Sets

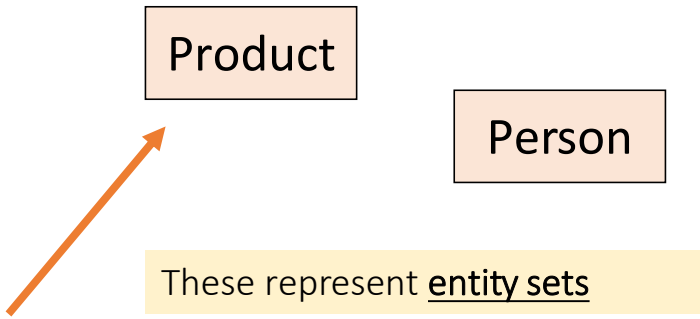
- Entity: An object in the world that can be distinguished from other objects
- Entity set: A set of similar entities.

Examples: Entity sets

PERSON	Example : Student, Employee
PLACE	Example : Country, Branch
OBJECT	Example : Product, Building
EVENT	Example : Student Registration, Payment
CONCEPT	Example : Course, Order

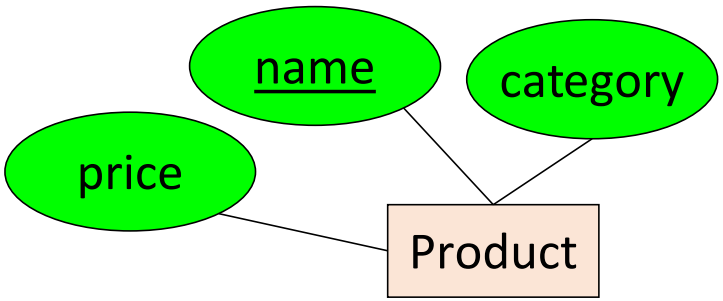
Entities and Entity Sets

- Entity set → Class or type
  - Nouns in the problem domain → entity sets
- Entity → Object of a class
- Only Entity sets are modeled and not the entities
- Entity sets are drawn as rectangles in the E/R diagram



Attributes

- Attributes → Used to describe an entity
- All entities in the set have the same attributes
- An attribute contains single piece of information (and not a list of data)
- Attributes are represented by ovals attached to an entity set

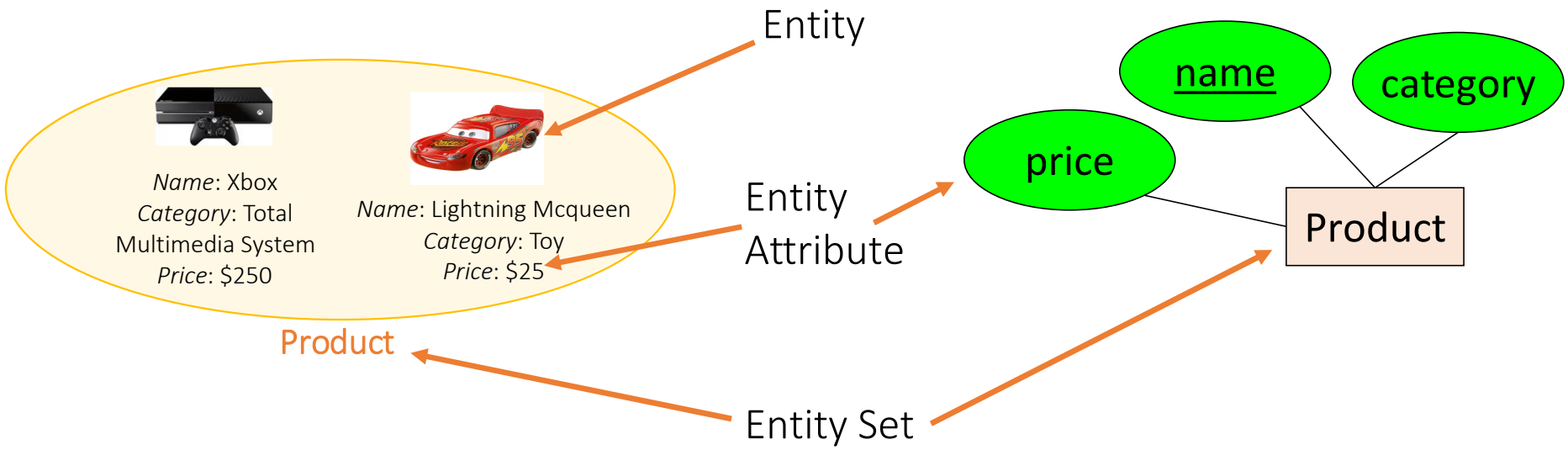


Shapes are important. Colors are not.

Entities, Entity Sets and Attributes

Example:

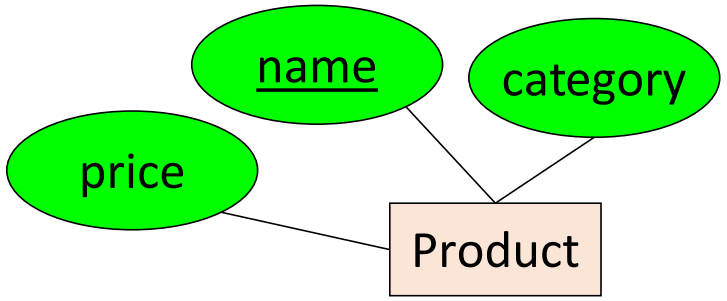
Entities are **not** explicitly represented in E/R diagrams!



Keys

- A *key* is a **minimal** set of attributes that uniquely identifies an entity.

Denote elements of the primary key by underlining.

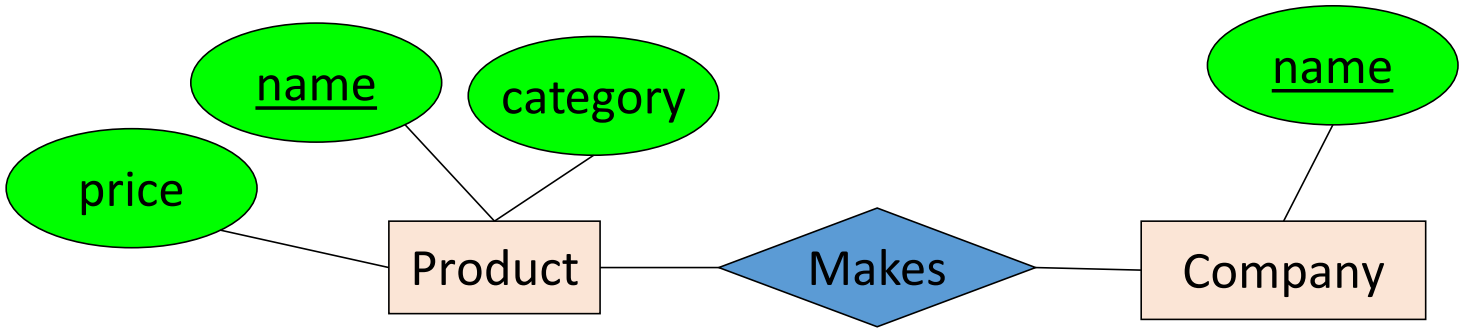


Here, {name, category} is not a key (it is not *minimal*).

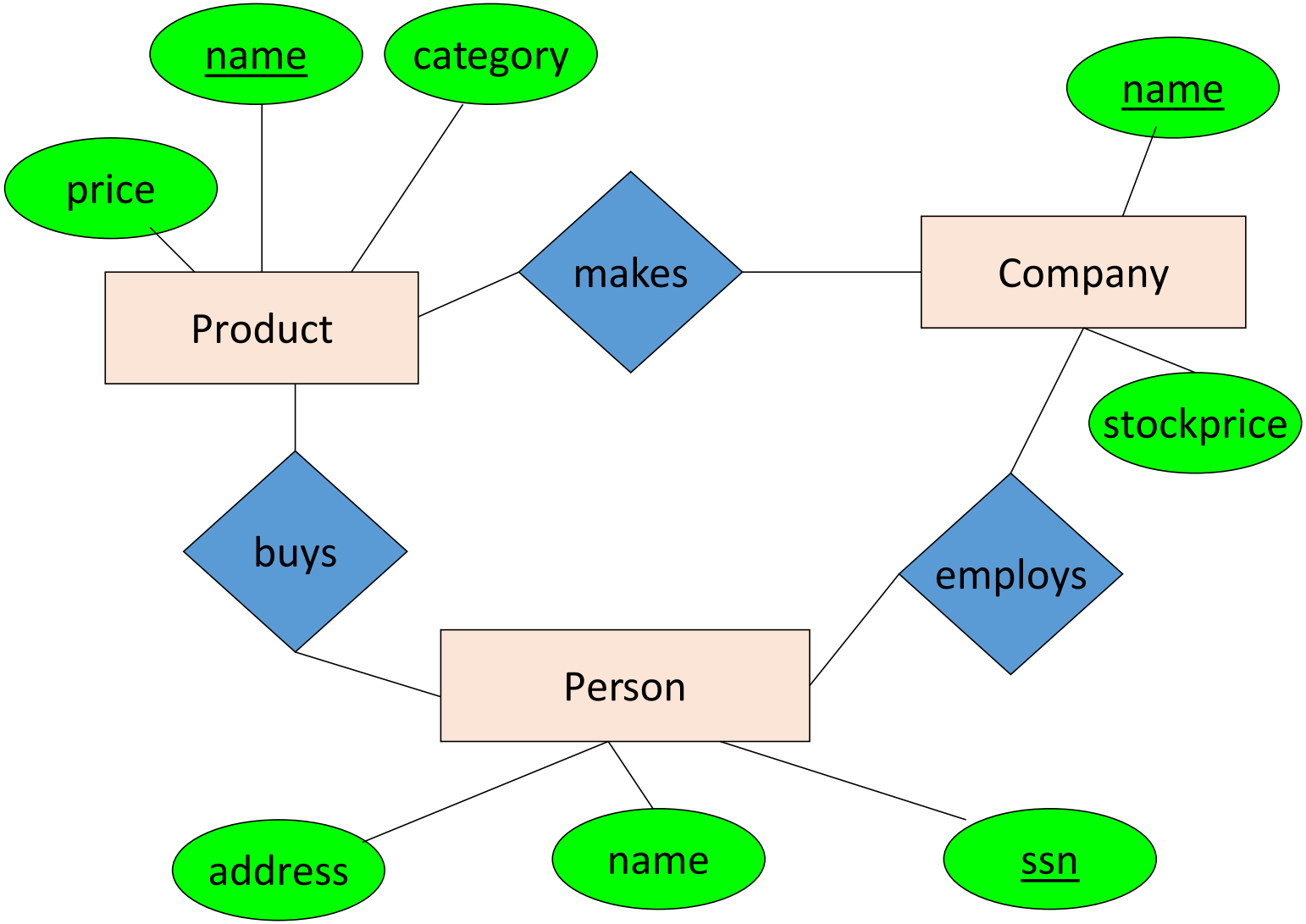
The E/R model forces us to designate a single primary key, though there may be multiple candidate keys

The R in E/R Diagram: **Relationships**

- A **relationship** is association among two or more entities
- Relationships are drawn using diamond

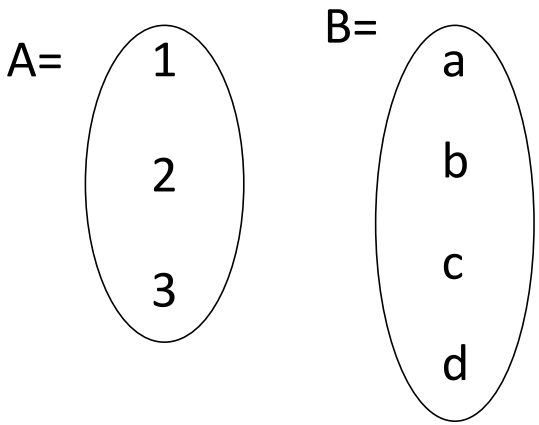


- Verbs in the problem domain → relationship or relationship set


name
category
price
Product
makes
Company
name
stockprice
buys
employs
Person
address
name
ssn

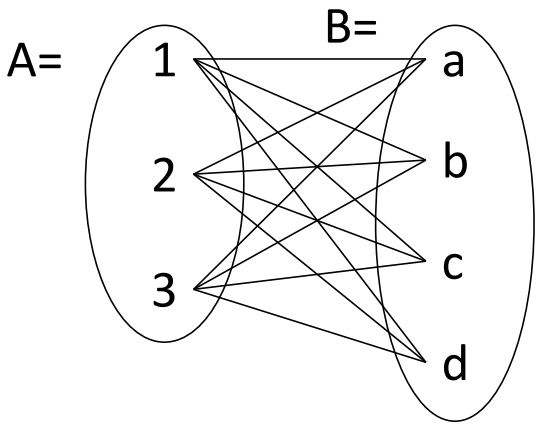
What is a Relationship?

- ***A mathematical definition:***
  - Let A, B be sets
    - *A={1,2,3}, B={a,b,c,d}*



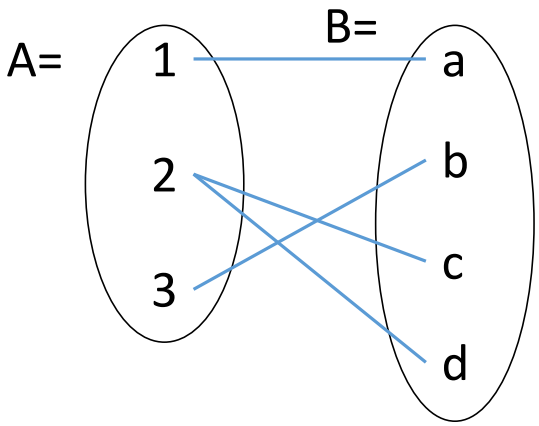
What is a Relationship?

- ***A mathematical definition:***
  - Let A, B be sets
    - *A={1,2,3}, B={a,b,c,d}*
  - A x B (the ***cross-product***) is the set of all pairs (a,b)
    - *$A \times B$ = {(1,a), (1,b), (1,c), (1,d), (2,a), (2,b), (2,c), (2,d), (3,a), (3,b), (3,c), (3,d)}*



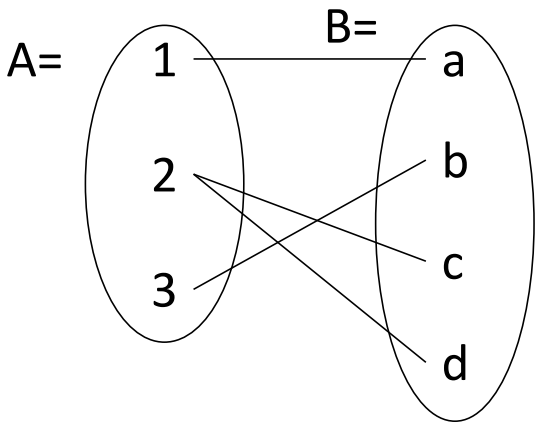
What is a Relationship?

- ***A mathematical definition:***
  - Let A, B be sets
    - *A={1,2,3}, B={a,b,c,d}*
  - A x B (the ***cross-product***) is the set of all pairs (a,b)
    - *$A \times B$ = {(1,a), (1,b), (1,c), (1,d), (2,a), (2,b), (2,c), (2,d), (3,a), (3,b), (3,c), (3,d)}*
  - **We define a relationship to be a subset of A x B**
    - *R = {(1,a), (2,c), (2,d), (3,b)}*

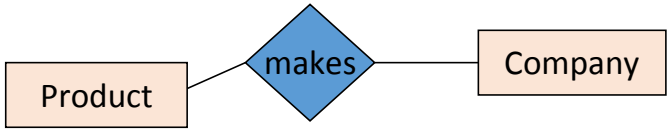


What is a Relationship?

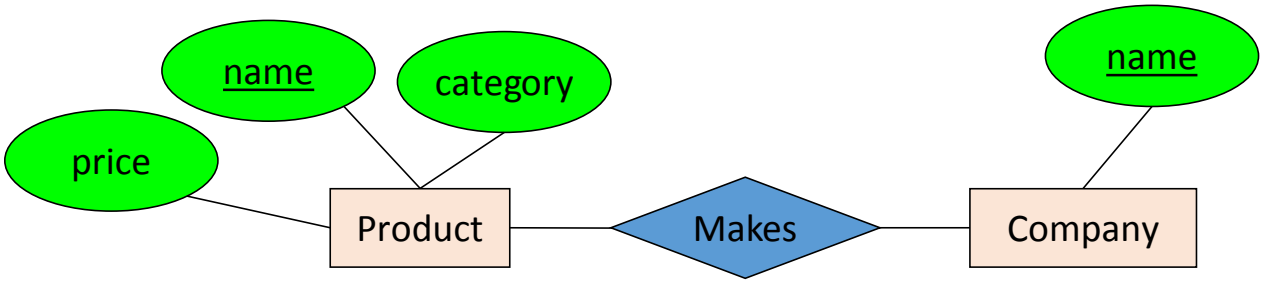
- ***A mathematical definition:***
  - Let A, B be sets
  - A x B (the ***cross-product***) is the set of all pairs
  - A relationship is a subset of A x B



- **Makes** is a relationship-it is a ***subset*** of **Product** × **Company**:



What is a Relationship?



A relationship between **entity sets P and C** is a ***subset of all possible pairs of entities in P and C,*** with tuples uniquely identified by ***P and C's keys***

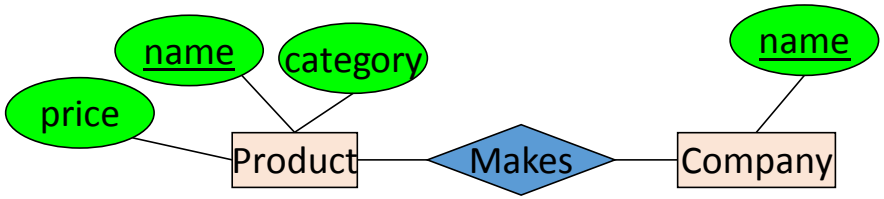
What is a Relationship?

Company

name
GizmoWorks
GadgetCorp

Product

name	category	price
Gizmo	Electronics	$9.99
GizmoLite	Electronics	$7.50
Gadget	Toys	$5.50



A relationship between entity sets P and C is a *subset of all possible pairs of entities in P and C*, with tuples uniquely identified by *P and C's keys*

What is a Relationship?

Company

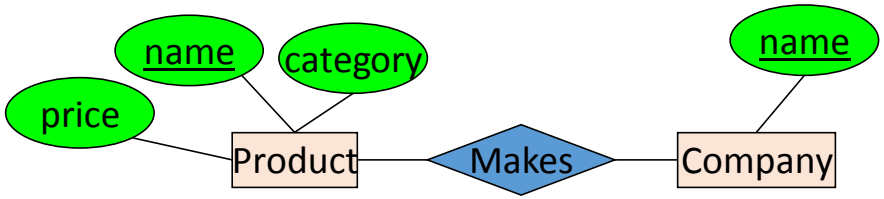
name
GizmoWorks
GadgetCorp

Product

name	category	price
Gizmo	Electronics	$9.99
GizmoLite	Electronics	$7.50
Gadget	Toys	$5.50

Company C × Product P

C.name	P.name	P.category	P.price
GizmoWorks	Gizmo	Electronics	$9.99
GizmoWorks	GizmoLite	Electronics	$7.50
GizmoWorks	Gadget	Toys	$5.50
GadgetCorp	Gizmo	Electronics	$9.99
GadgetCorp	GizmoLite	Electronics	$7.50
GadgetCorp	Gadget	Toys	$5.50



A relationship between entity sets P and C is a *subset of all possible pairs of entities in P and C*, with tuples uniquely identified by *P and C's keys*

What is a Relationship?

Company

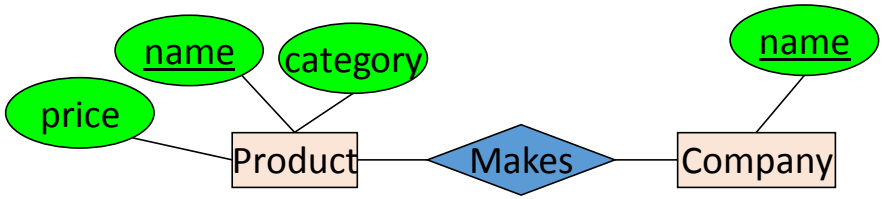
name
GizmoWorks
GadgetCorp

Product

name	category	price
Gizmo	Electronics	$9.99
GizmoLite	Electronics	$7.50
Gadget	Toys	$5.50

Company C × Product P

C.name	P.name	P.category	P.price
GizmoWorks	Gizmo	Electronics	$9.99
GizmoWorks	GizmoLite	Electronics	$7.50
GizmoWorks	Gadget	Toys	$5.50
GadgetCorp	Gizmo	Electronics	$9.99
GadgetCorp	GizmoLite	Electronics	$7.50
GadgetCorp	Gadget	Toys	$5.50



Makes

C.name	P.name
GizmoWorks	Gizmo
GizmoWorks	GizmoLite
GadgetCorp	Gadget

A relationship between entity sets P and C is a *subset of all possible pairs of entities in P and C*, with tuples uniquely identified by *P and C's keys*

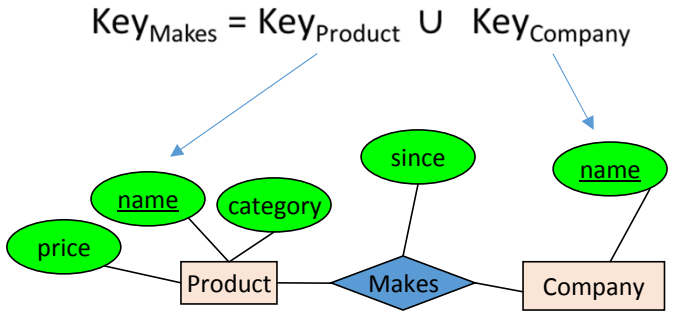
What is a Relationship?

- There can only be **one relationship for every unique combination of entities**

This follows from our mathematical definition of a relationship- it's a SET!

- This also means that **the relationship is uniquely determined by the keys of its entities**

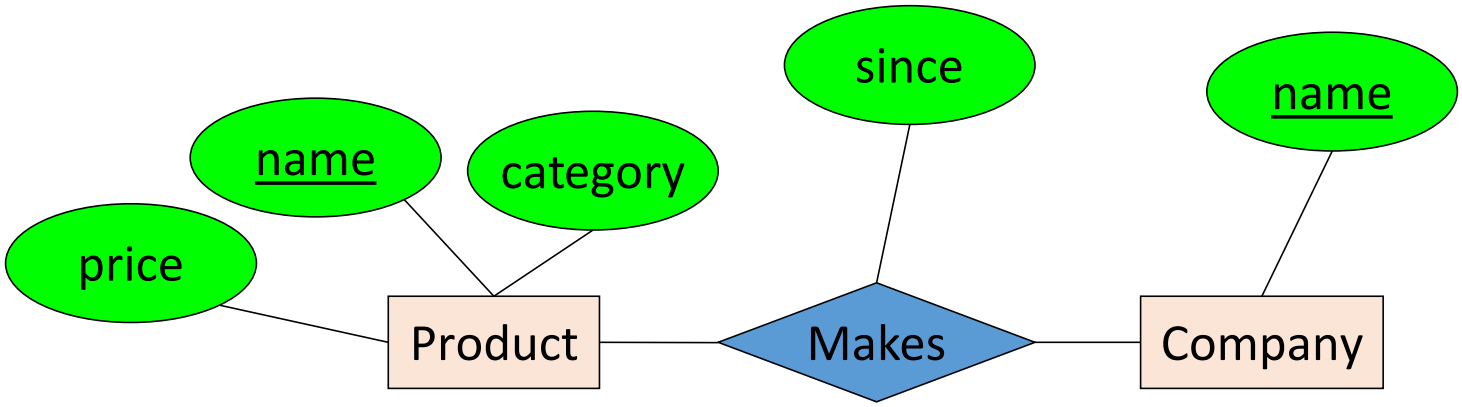
- *Example: the key for Makes (to right) is {Product.name, Company.name}*



Why does this make sense?

Relationships and Attributes

- Relationships may have attributes as well.



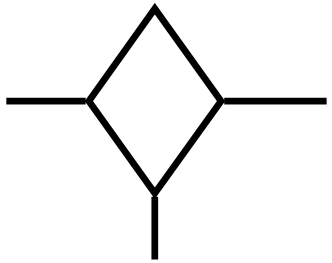
For example: "since" records when company started making a product

Note: *"since"* is implicitly unique per pair here! Why?

Note #2: Why not "how long"?

Tools at a glance

Entities (‘entity sets’)

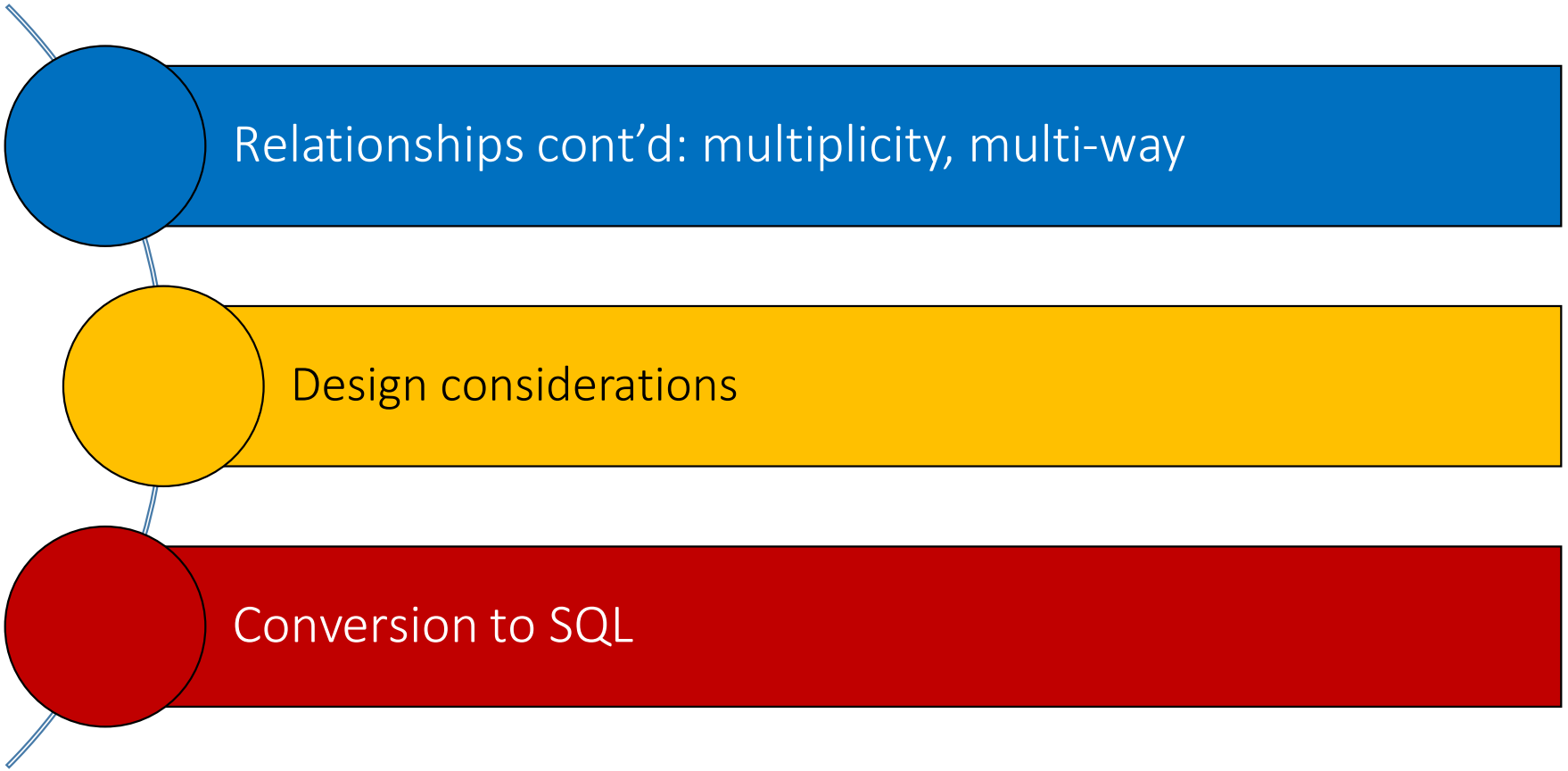


Relationships (‘rel. sets’) and mapping constraints

Attributes

2. E/R Design Considerations

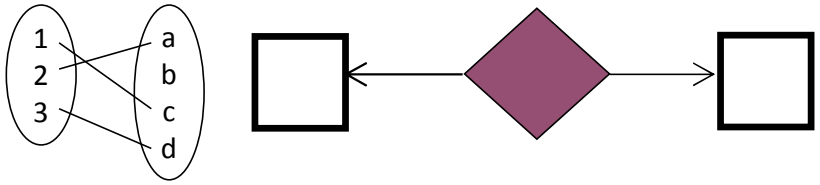
What you will learn in this section



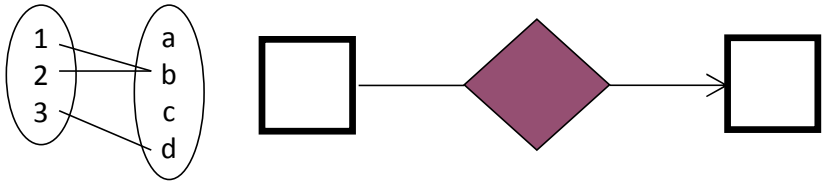
Multiplicity of E/R Relationships

Multiplicity of E/R Relationships

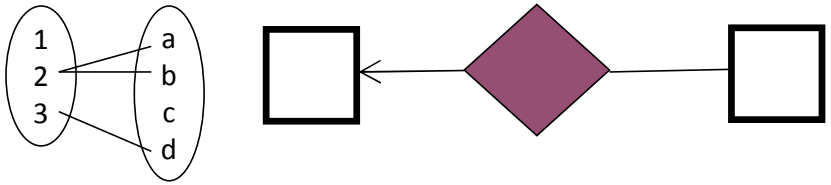
One-to-one:



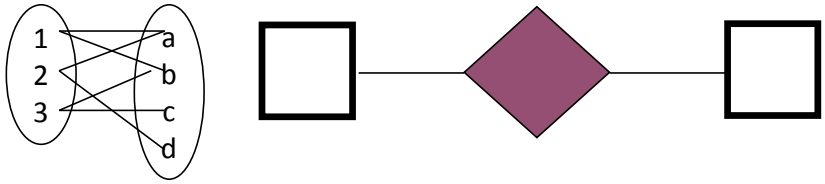
Many-to-one:



One-to-many:



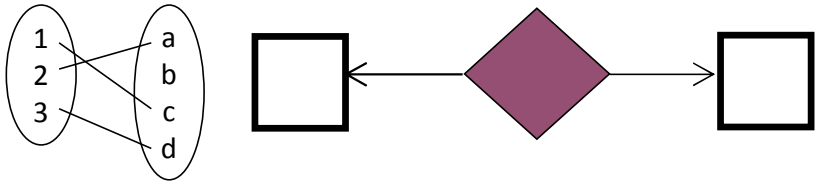
Many-to-many:



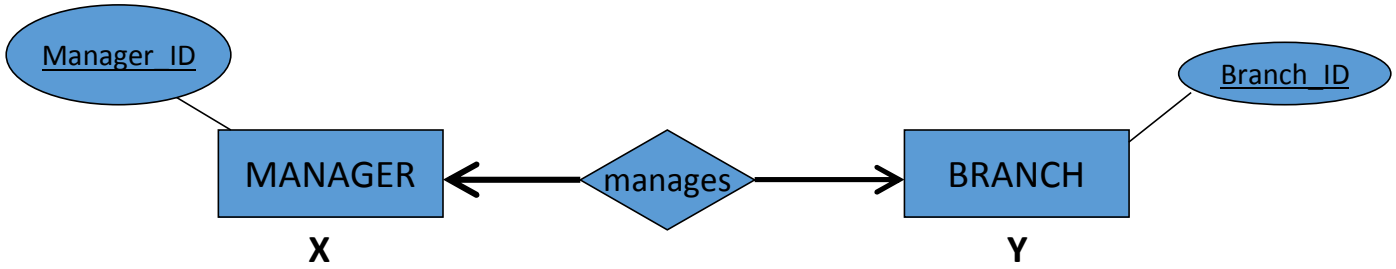
Indicated using arrows

Example: One to One Relationships (1:1)

One-to-one:



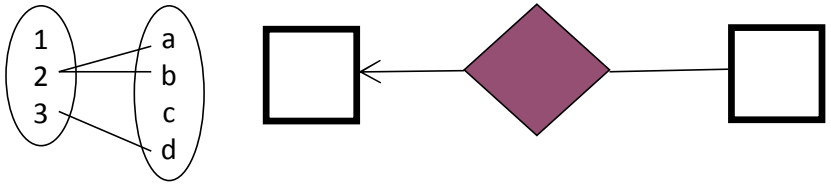
Def : Maximum one X for each Y and one Y for each X



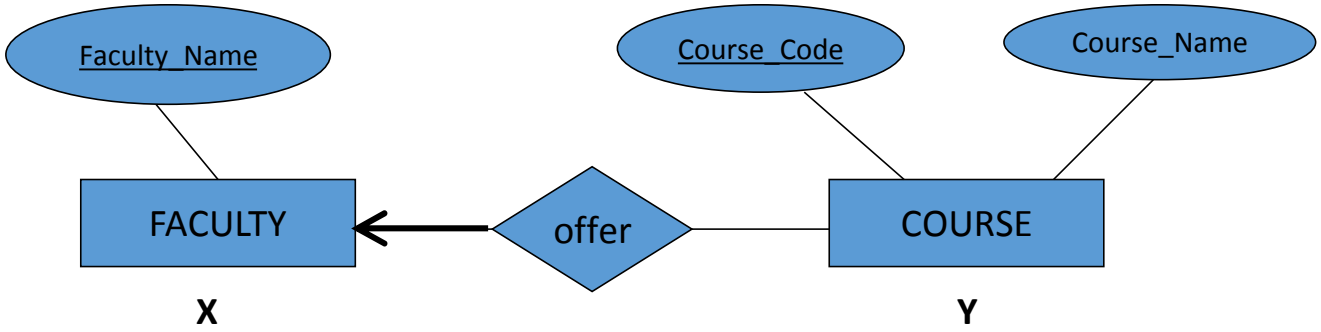
An employee (manager) can manage only one branch at one time and each branch has only one manager.

Example: One to Many Relationships (1:M)

One-to-many:



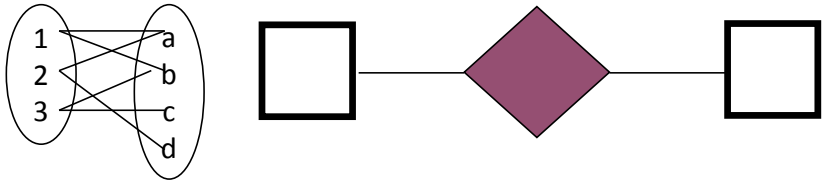
Def : Maximum one X for each Y but possibly many Y for each X



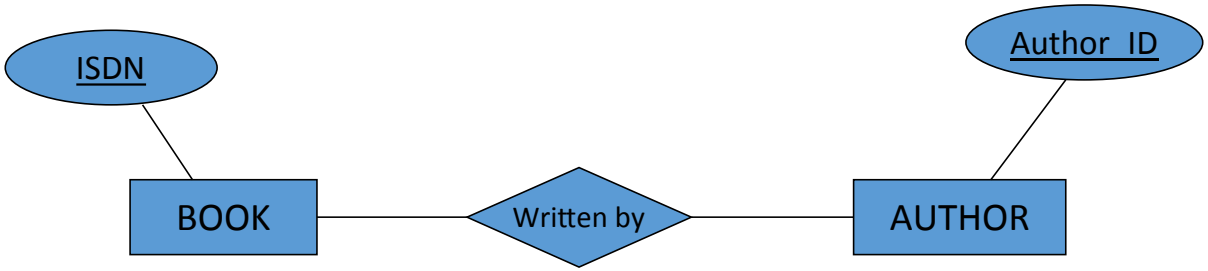
At university, one faculty offers many courses for students but one course is offered by one faculty only.

Example: Many to Many Relationships (M:N)

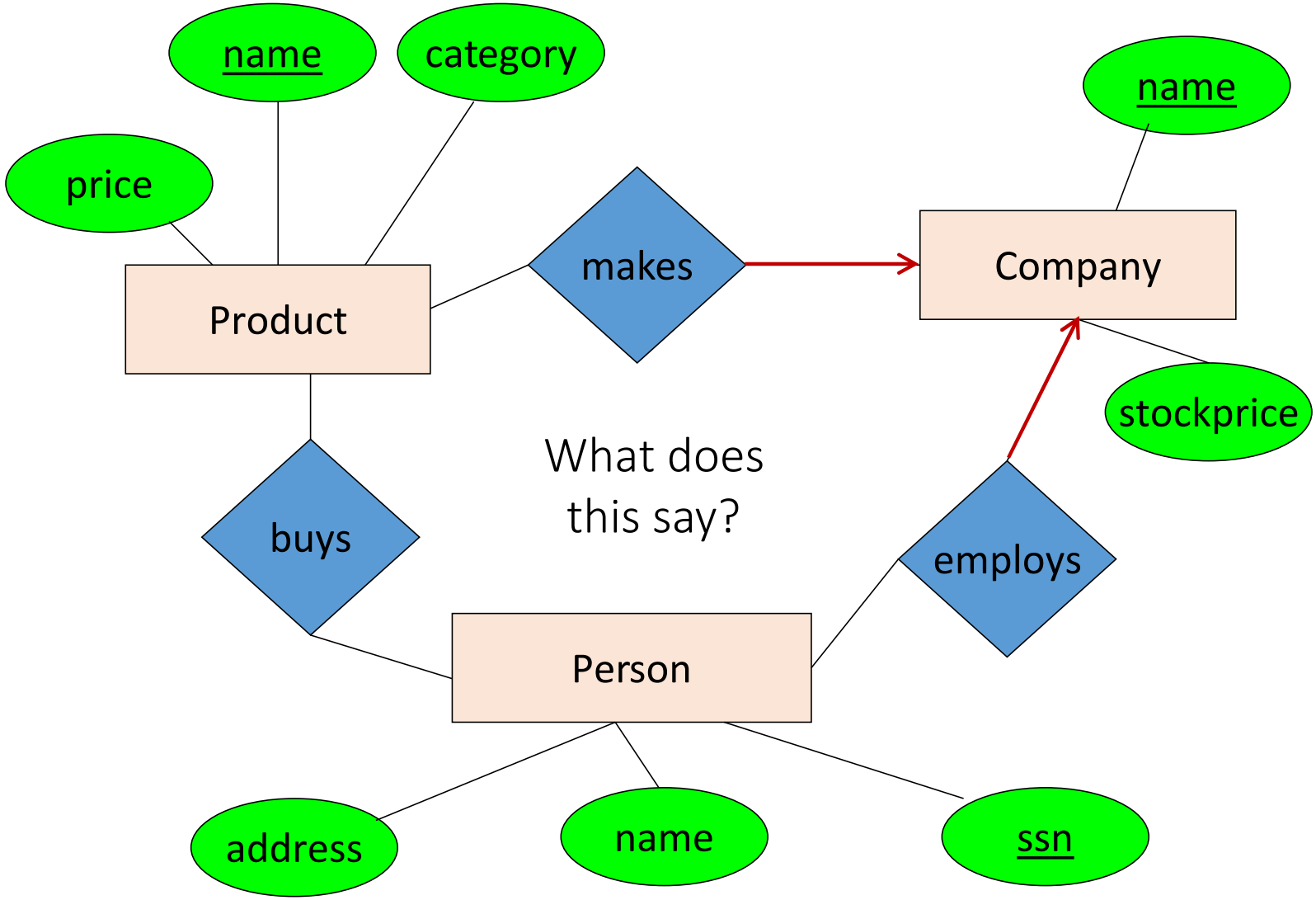
Many-to-many:



Def : Possibly many X's for each Y and many Y's for each X

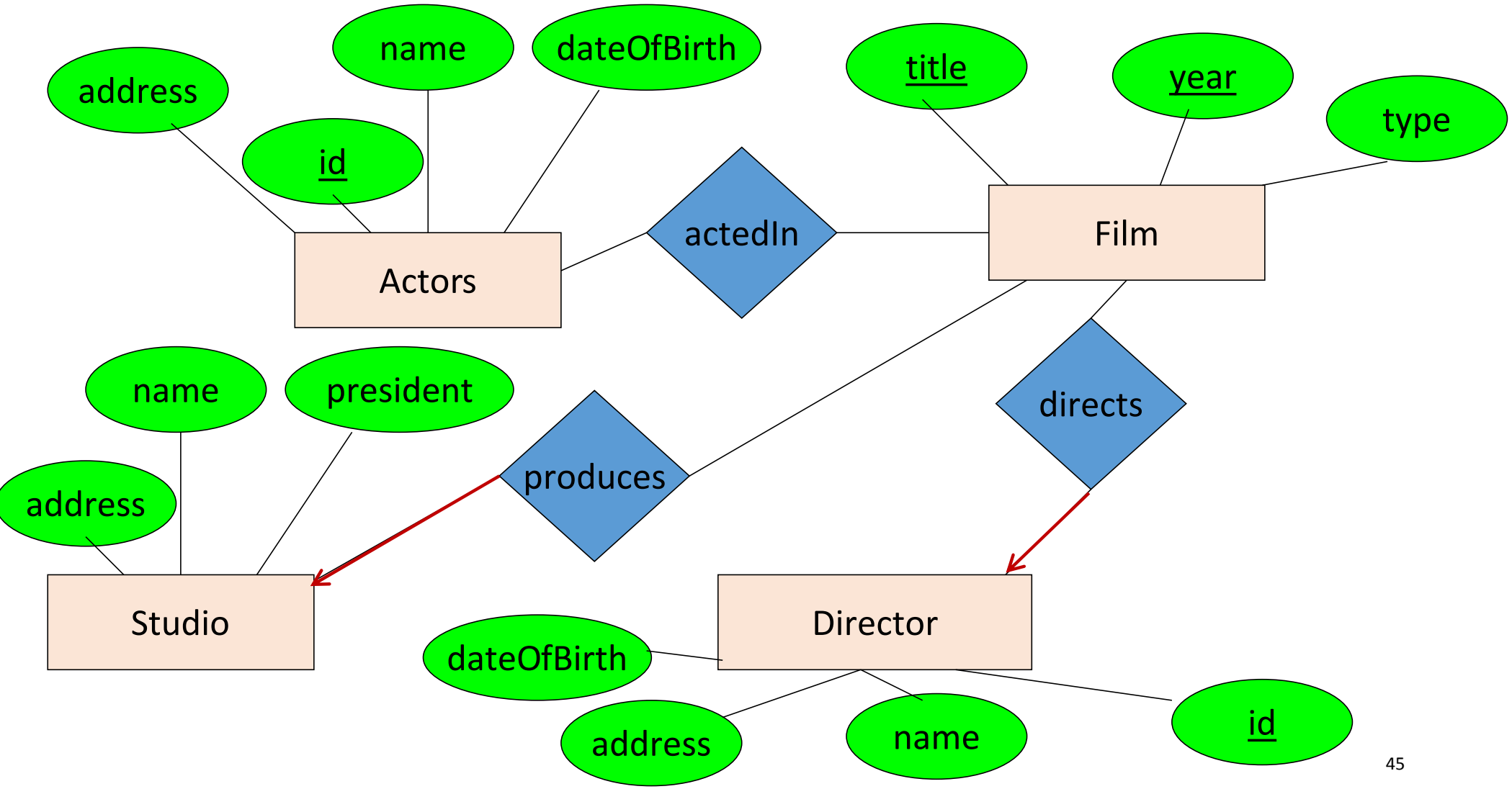


A book may have more than one author and an author may write more than one book.

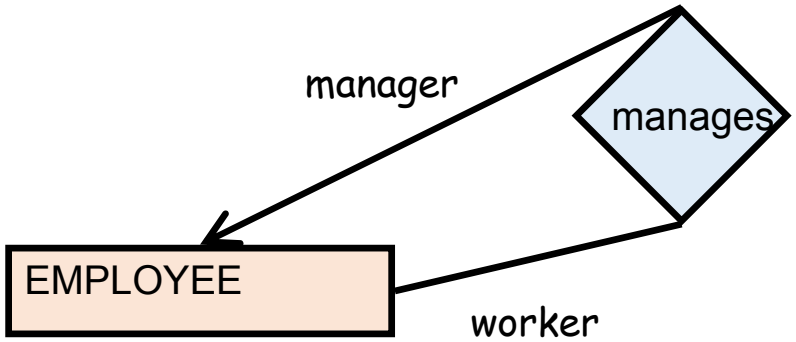

name
category
price
Product
makes
Company
name
stockprice
What does this say?
buys
employs
Person
address
name
ssn

Activity in the class

- For actors and directors, we want to store their name, a unique identification number, address and birthday (why not age?)
- For films, we want to store the title, year of production and type (thriller, comedy, etc.)
- We want to know who directed and who acted in each film. Every film has one director. We store the salary of each actor for each film
- For Studio we want to store name, address and name of its president. A movie can be produced by only one studio.


name
dateOfBirth
address
id
title
year
type
Actors
actedIn
Film
name
president
directs
produces
address
Studio
Director
dateOfBirth
address
name
id

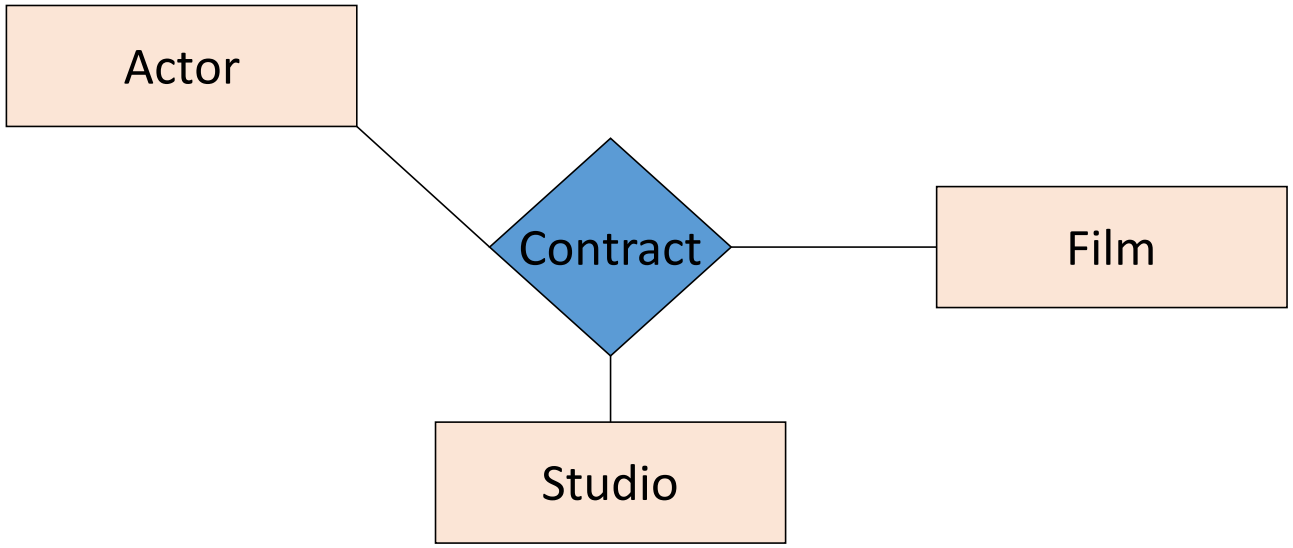
Self Relationships or Recursive Relationships


manager
manages
EMPLOYEE
worker

Multi-way Relationships

Multi-way Relationships

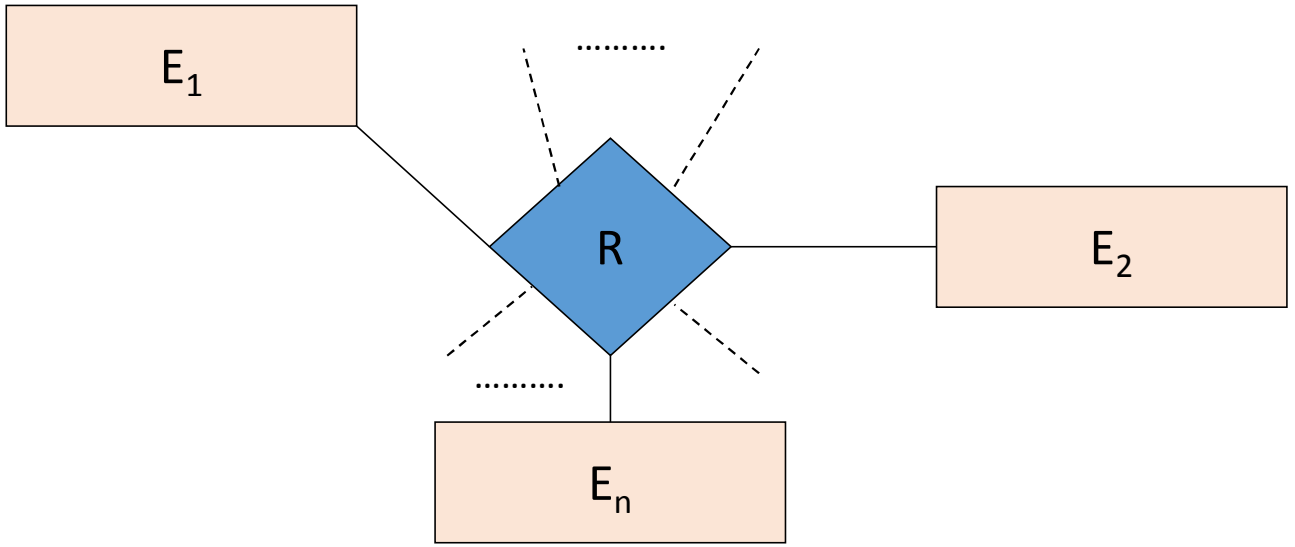
How do we model that a studio has made a contract with a particular actor to act in a particular movie?



NB: Can still model as a mathematical set (how?)

Formal mathematical definition

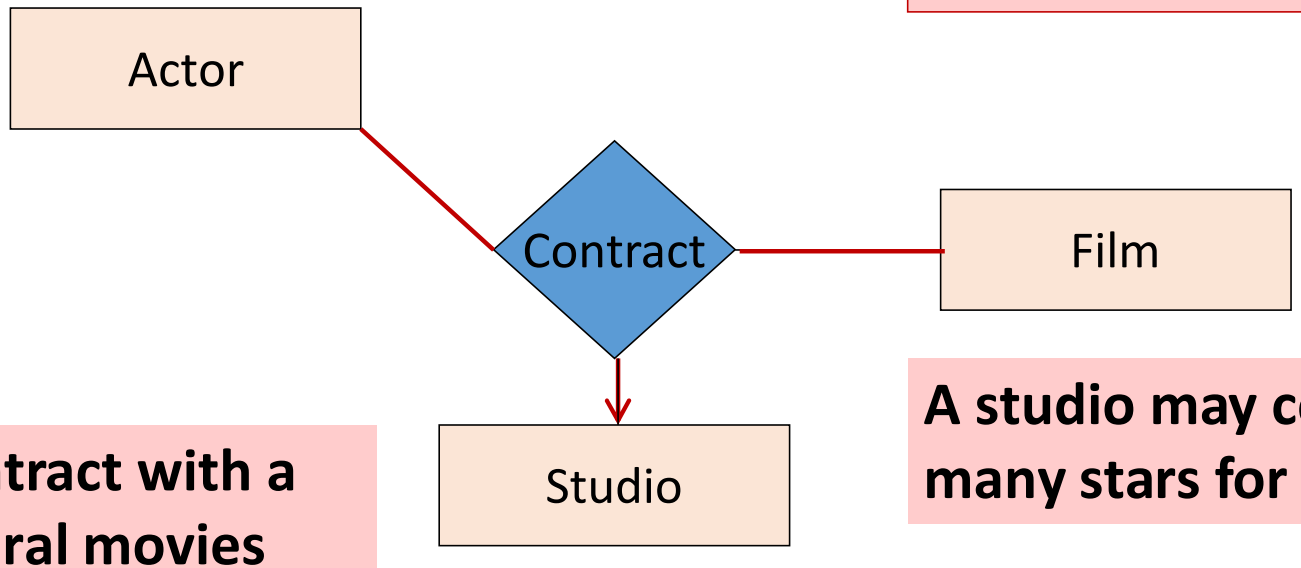
- An *n*-ary relationship set *R* involves exactly *n* entity sets: E_1, …, E_n.
- Each relationship in *R* involves exactly *n* entities: e_1 in E_1, …, e_n in E_n
- Formally, $R \subseteq E_1 x \ldots x E_n$



Arrows in Multiway Relationships

Q: What does the arrow mean ?

For a particular star and movie there is only one studio under the contract

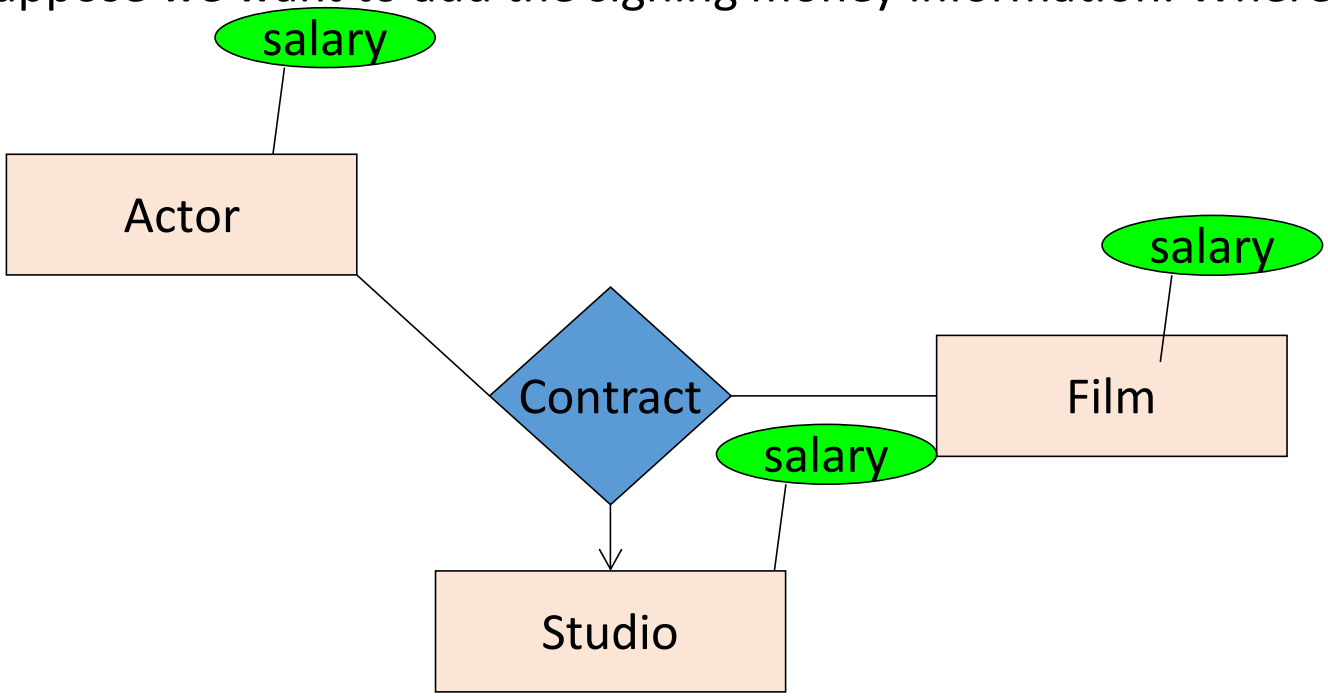


A star may contract with a studio for several movies

A studio may contract with many stars for a movie

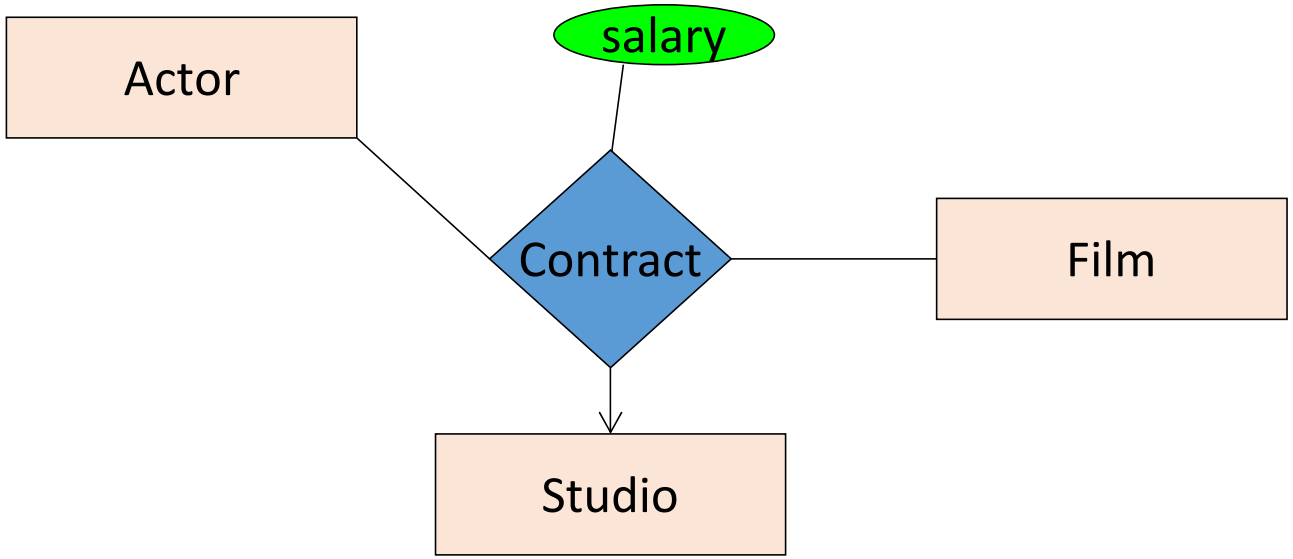
Attributes in Relationships

Q: Suppose we want to add the signing money information. Where do we put?

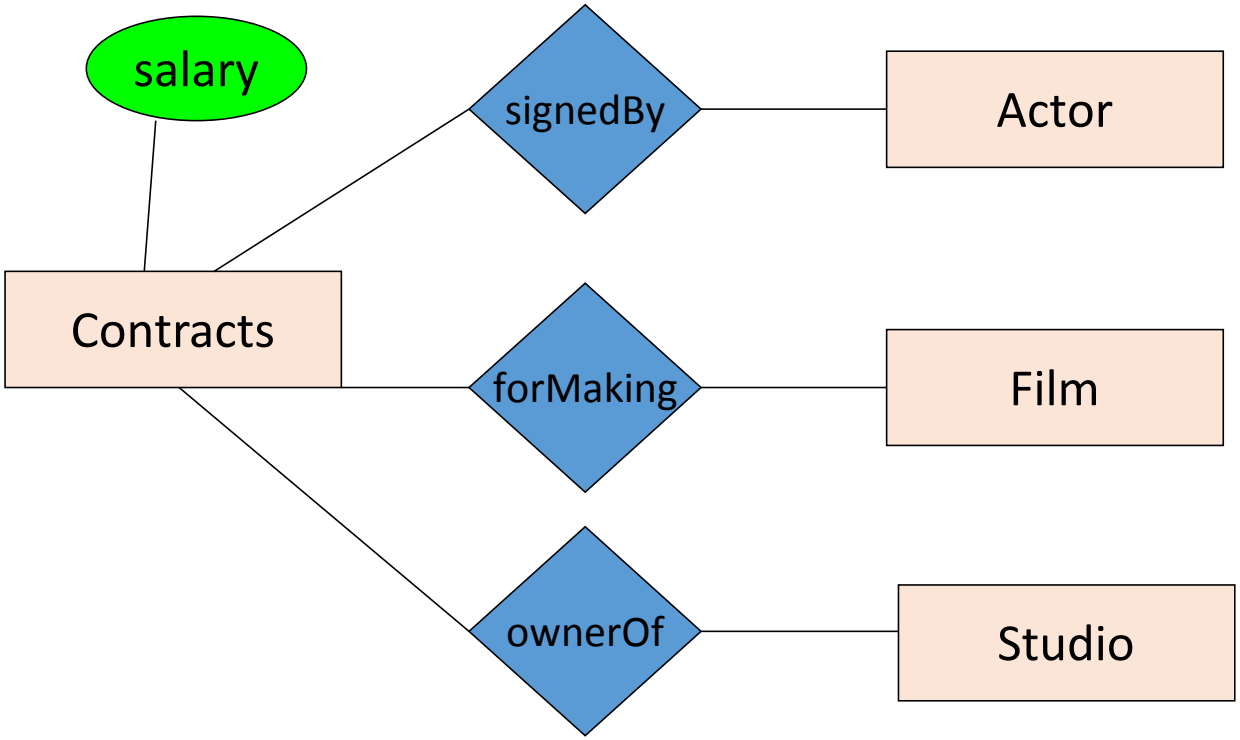


Attributes in Relationships: The correct way

Q: Suppose we want to add the signing money information. Where do we put?

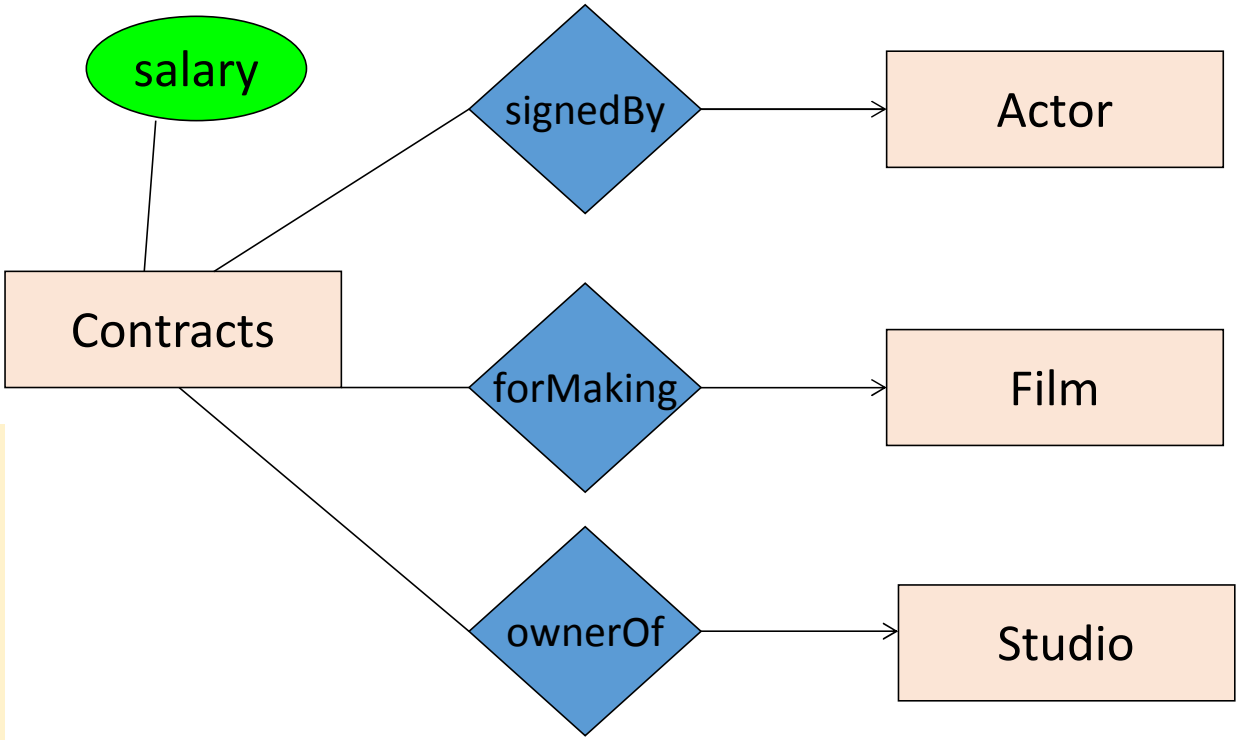


Converting Multi-way Relationships to Binary Relations



From what we had on previous slide to this - what did we do?

Converting Multi-way Relationships to New Entity + Binary Relationships



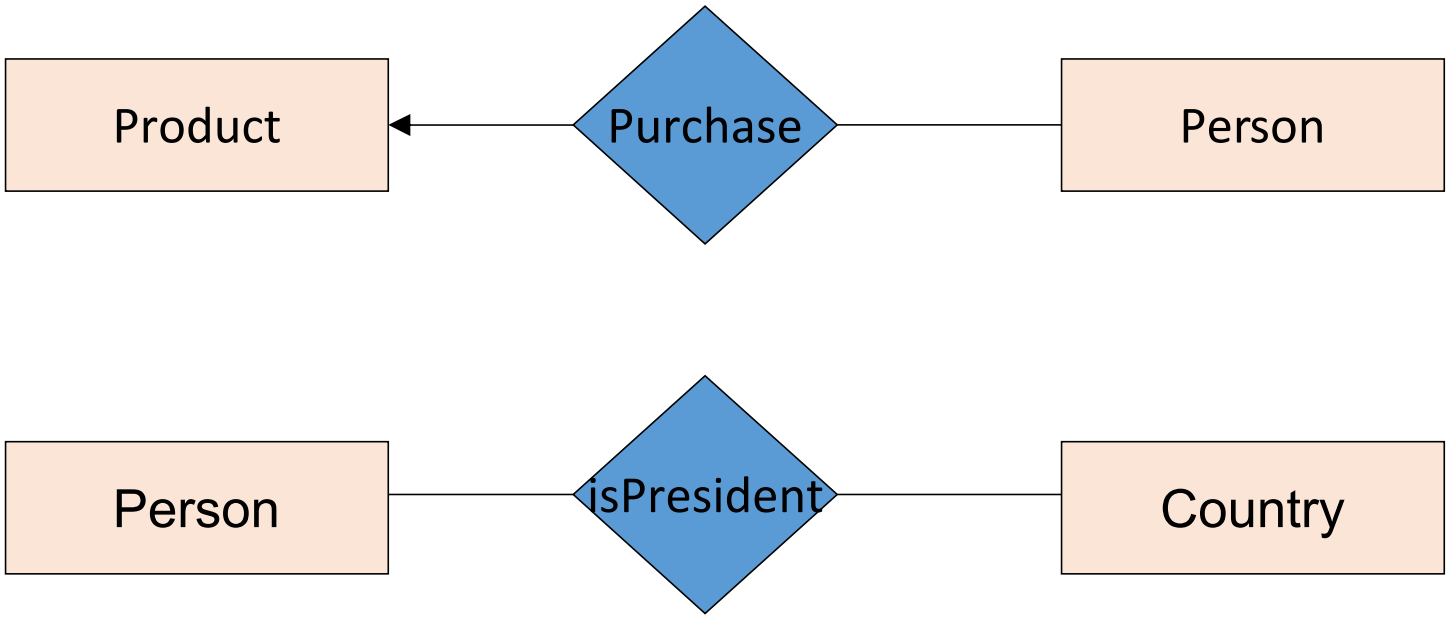
Side note: What arrows should be added here? Are these correct?

For a particular contract there can be only one star, one movie and one studio. But an actor or a movie or a studio may have several contracts

3. Design Principles

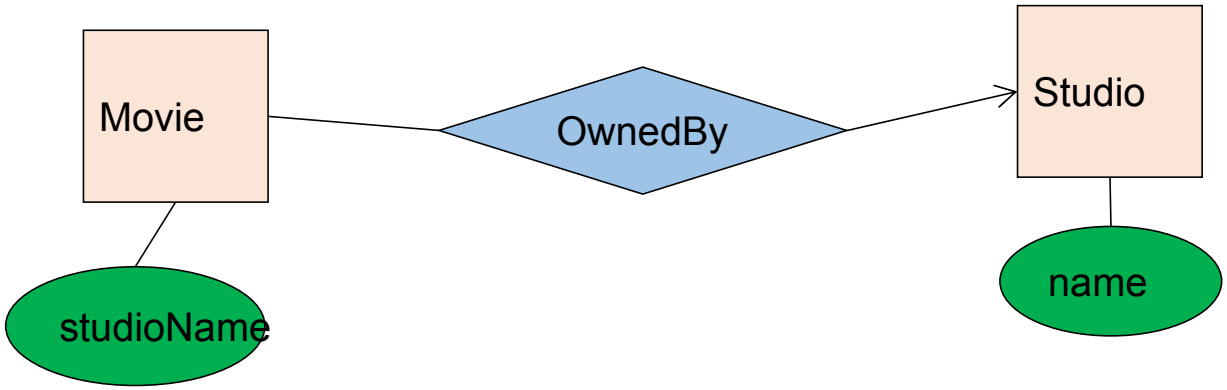
Design Principles

What's wrong with these examples?



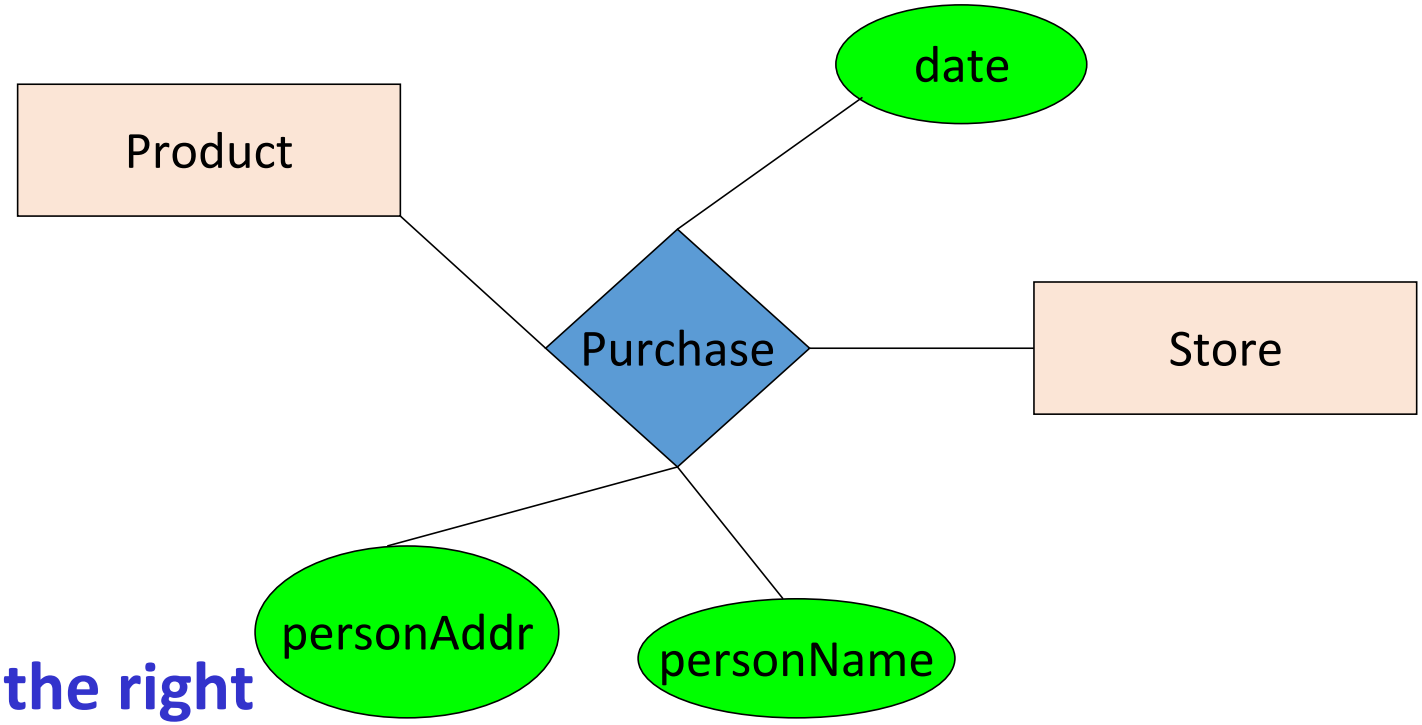
Moral: be faithful!

Design Principles: What's Wrong?



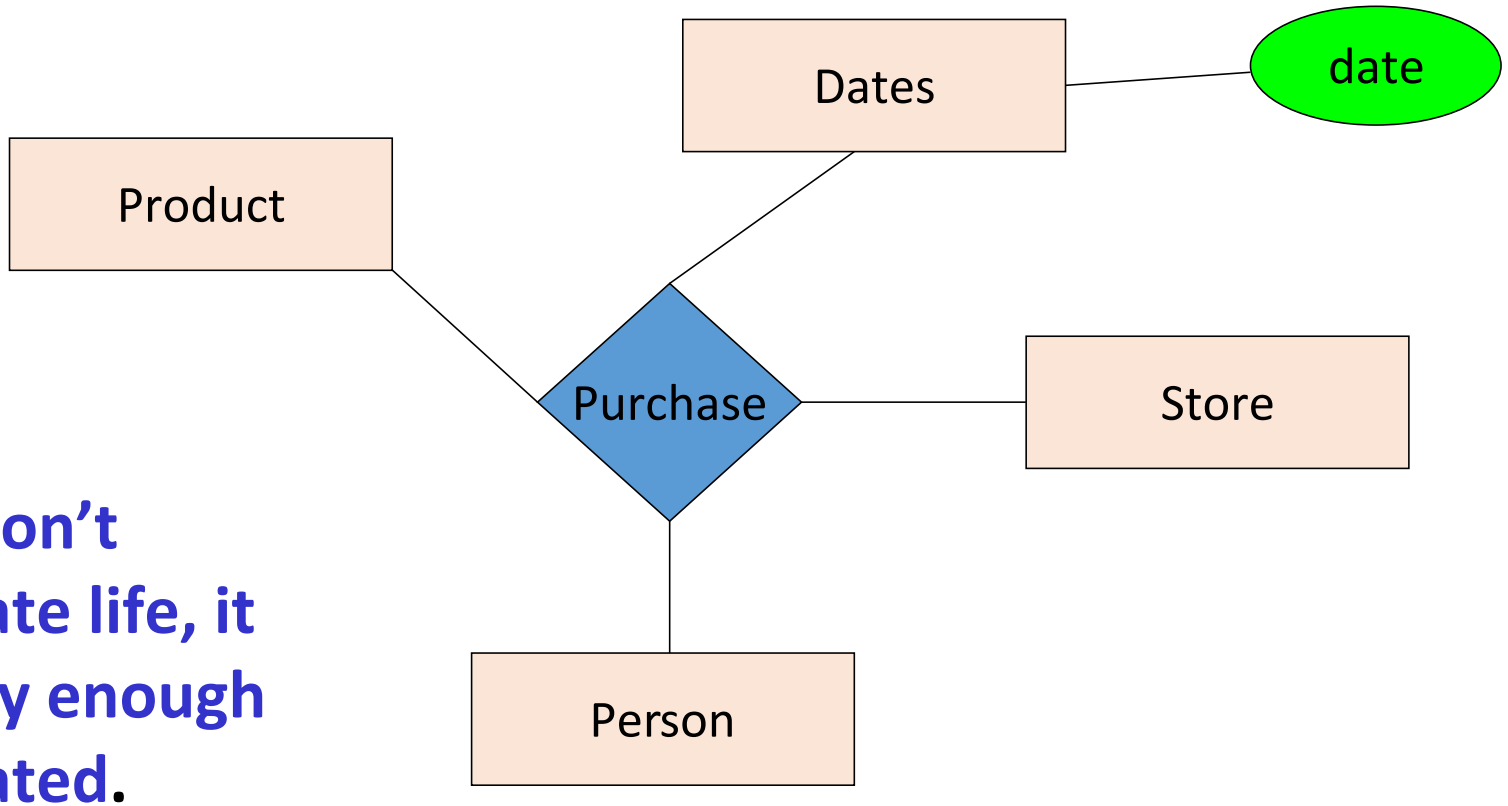
Moral: should not model the same information in multiple ways

Design Principles: What's Wrong?



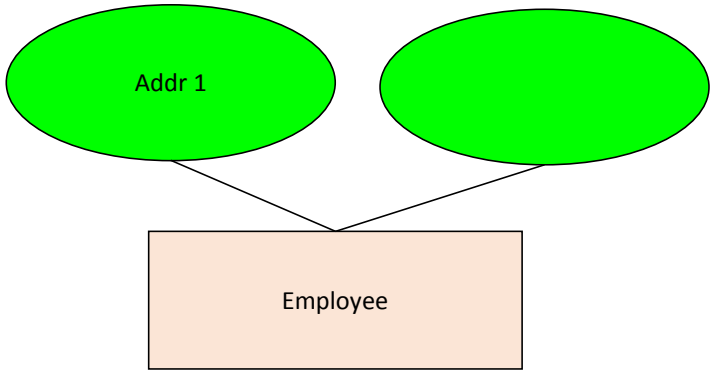
Moral: pick the right kind of elements (entities/relationships).

Design Principles: What's Wrong?



Moral: don't complicate life, it is already enough complicated.

Design Principles: Entity vs. Attribute

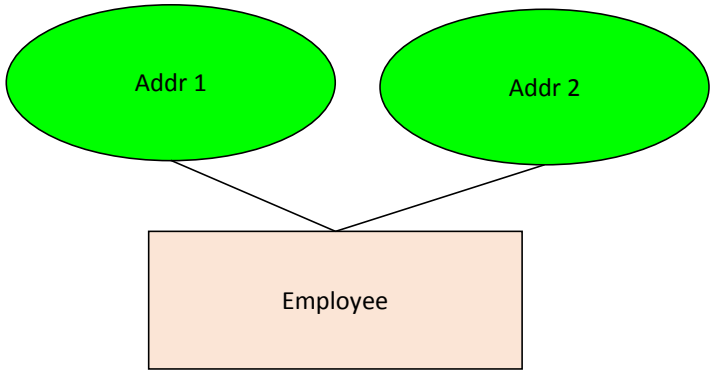


How do we handle employees with multiple addresses here?

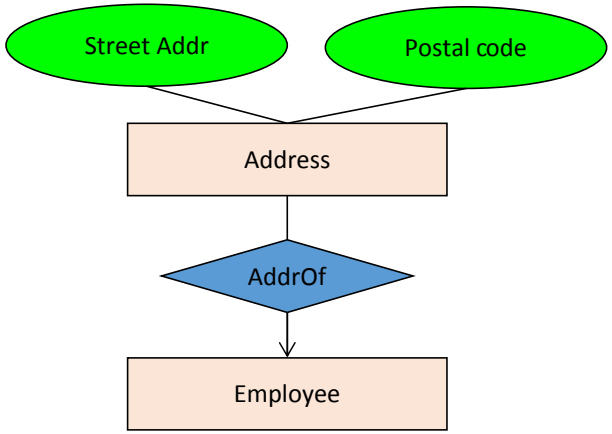
How do we handle addresses where internal structure of the address (e.g. postal code, city, division) is useful?

Design Principles: Entity vs. Attribute

Should address (A) be an attribute?



Or (B) be an entity?



In general, when we want to record several values, we choose new entity

From E/R Diagrams to Relational Schema

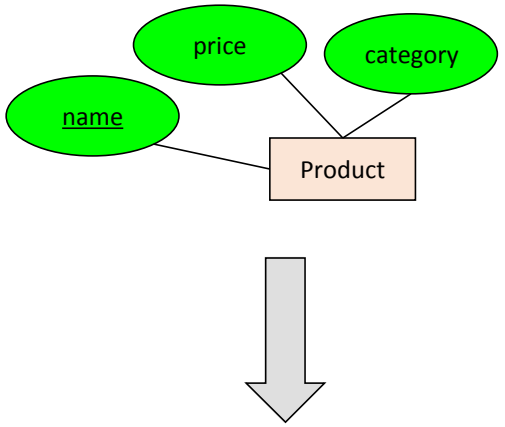
From E/R Diagrams to Relational Schema

- Key concept:

  Both ***Entity sets*** and ***Relationships*** become relations (tables in RDBMS)

From E/R Diagrams to Relational Schema

- An entity set becomes a relation (collection of tuples / table)
  - Each tuple is one entity
  - Each tuple is composed of the entity's attributes, and has the same primary key

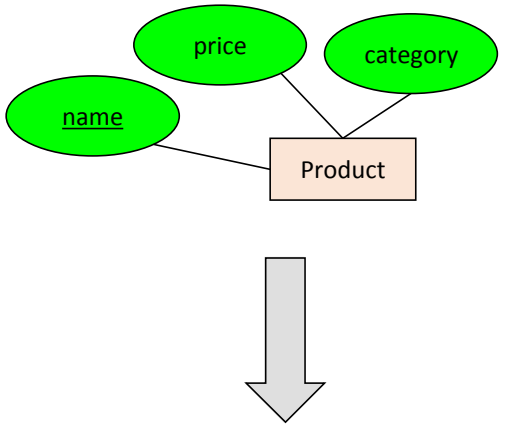


Product

name	price	category
Gizmo1	99.99	Camera
Gizmo2	19.99	Edible

From E/R Diagrams to Relational Schema

```
CREATE TABLE Product(
  name     CHAR(50) PRIMARY KEY,
  price    FLOAT,
  category VARCHAR(30)
)
```



Product

name	price	category
Gizmo1	99.99	Camera
Gizmo2	19.99	Edible

From E/R Diagrams to Relational Schema

Converting **Many-to-Many** Relationships to Relational Schema

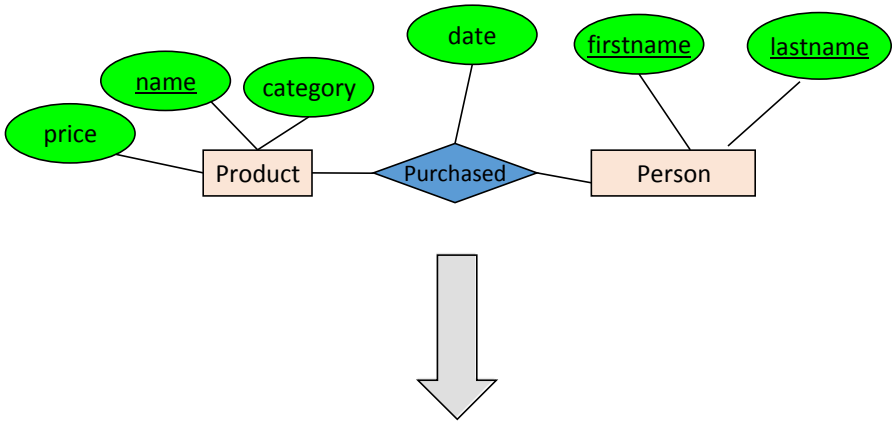
From E/R Diagrams to Relational Schema

- A relationship between entity sets $A_1, \ldots, A_N$

also becomes a set of tuples / a table

  - Each row/tuple is one relation, i.e. one unique combination of entities $(a_1,\ldots,a_N)$

  - Each row/tuple is
    - composed of the **union of the entity sets' key attributes**
    - has the entities' primary keys as foreign keys
    - has the union of the entity sets' keys as primary key

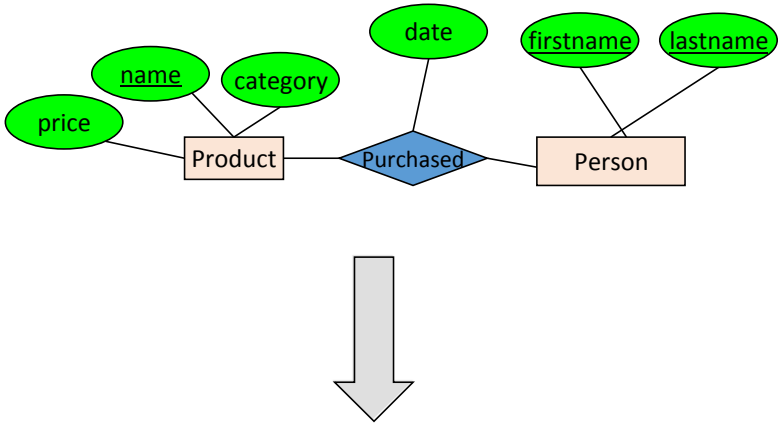


Purchased

name	firstname	lastname	date
Gizmo1	Bob	Joe	01/01/15
Gizmo2	Joe	Bob	01/03/15
Gizmo1	JoeBob	Smith	01/05/15

From E/R Diagrams to Relational Schema

```
CREATE TABLE Purchased(
  name      CHAR(50),
  firstname CHAR(50),
  lastname  CHAR(50),
  date      DATE,
  PRIMARY KEY (name, firstname, lastname),
  FOREIGN KEY (name)
            REFERENCES Product(name),
  FOREIGN KEY (firstname, lastname)
            REFERENCES Person(firstname, lastname)
)
```



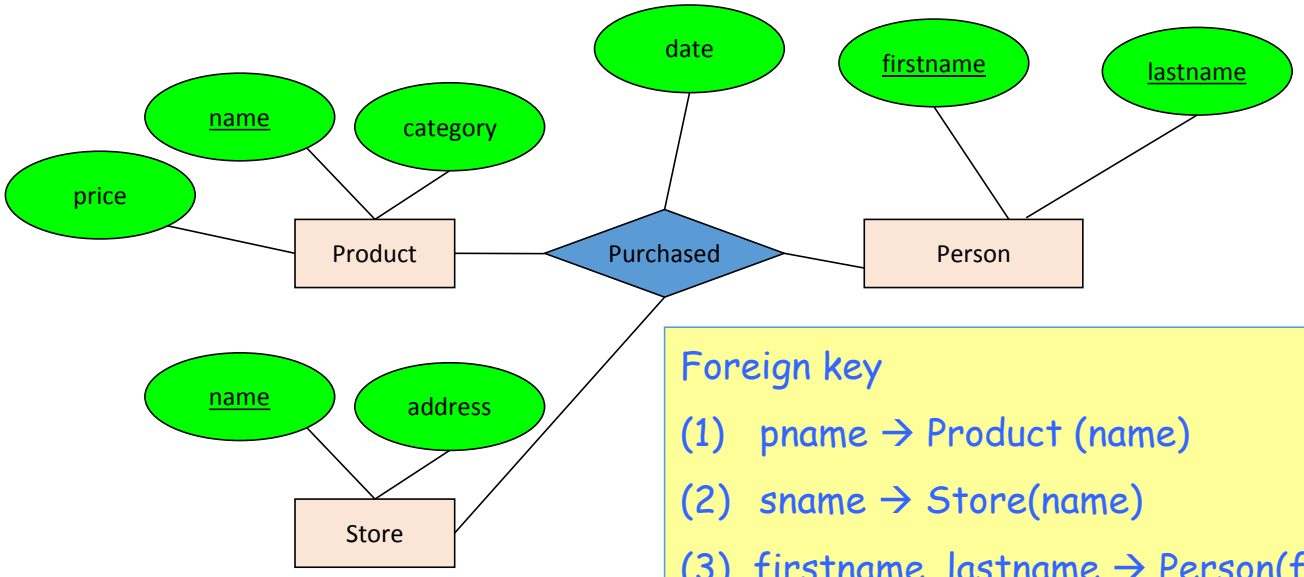
Purchased

name	firstname	lastname	date
Gizmo1	Bob	Joe	01/01/15
Gizmo2	Joe	Bob	01/03/15
Gizmo1	JoeBob	Smith	01/05/15

From E/R Diagram to Relational Schema

How do we represent this as a relational schema?

Purchased (pname, sname, firstname, lastname, date)



Foreign key

(1) pname → Product (name)

(2) sname → Store(name)

(3) firstname, lastname → Person(firstname, lastname)

From E/R Diagrams to Relational Schema

Converting One-to-Many Relationships to Relational Schema

From E/R Diagram to Relational Schema

- For many to one relations we have two options:

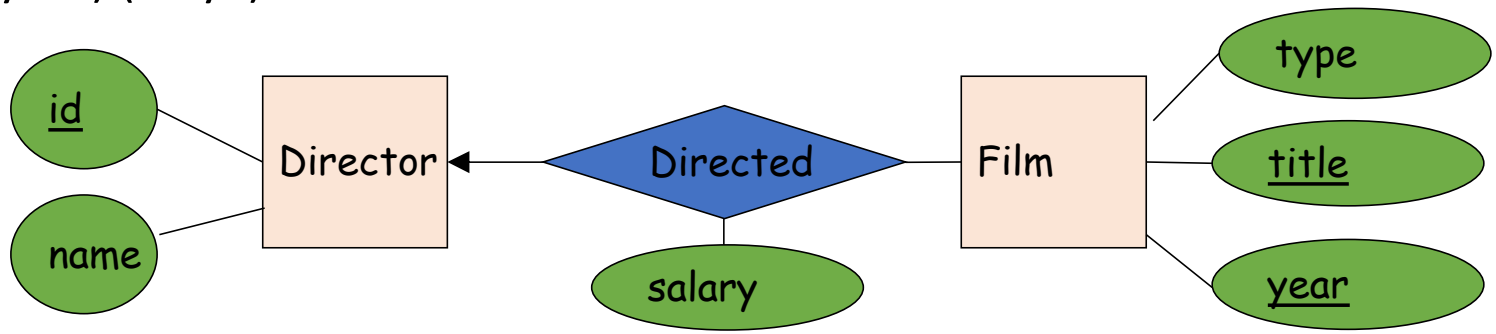
Option 1:

Same as without key constraints (3 tables), except that the primary key of Directed is now (title, year) (why?)

Film (<u>title</u>, <u>year</u>, type)

Director (<u>id</u>, name)

Directed (<u>title</u>, <u>year</u>, id, salary)



From E/R Diagram to Relational Schema

Film (<u>title</u>, <u>year</u>, type)

Director (<u>id</u>, name)

Directed (<u>title</u>, <u>year</u>, id, salary)

```
CREATE TABLE Film(
  title      CHAR(50),
  year       INT,
  type       CHAR(40),
  PRIMARY KEY (title, year)
)
```

```
CREATE TABLE Director(
  id         INT,
  name       CHAR(40),
  PRIMARY KEY (id)
)
```

```
CREATE TABLE Directed(
  title      CHAR(50),
  year       INT,
  id         INT,
  salary     FLOAT,
  PRIMARY KEY (title, year),
  FOREIGN KEY (title, year)
             REFERENCES Film (title, year),
  FOREIGN KEY (id)
             REFERENCES Director(id)
)
```

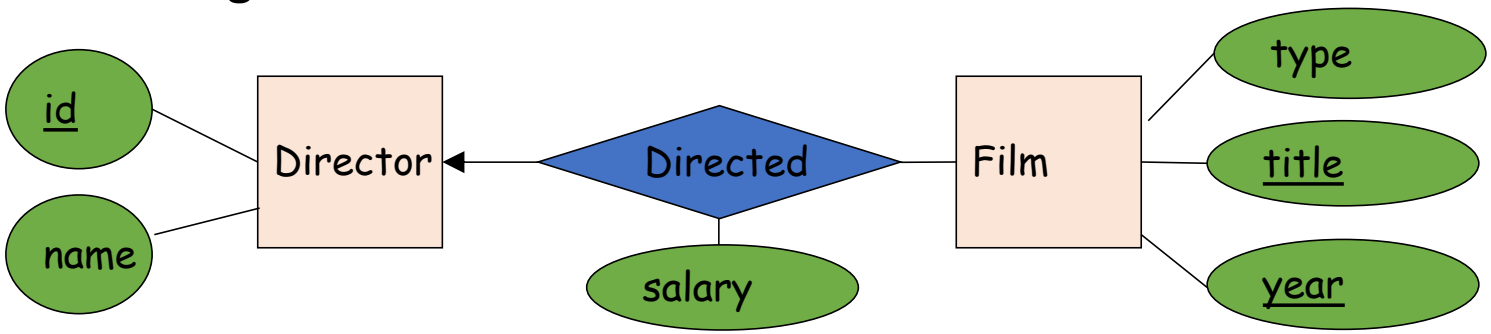
From E/R Diagram to Relational Schema

Film (<u>title</u>, <u>year</u>, type, id, salary)

Director (<u>id</u>, name)

Option 2:

- Do not create a table for the relationship
- Add information columns that would have been in the relationship's relation to the relation of the **entity which does not have the incoming arrow**



From E/R Diagram to Relational Schema

Film (title, year, type, id, salary)

Director (id, name)

```
CREATE TABLE Film(
  title      CHAR(50),
  year       INT,
  type       CHAR(40),
  id         INT,
   salary    FLOAT,
  PRIMARY KEY (title, year),
   FOREIGN KEY (id)
              REFERENCES Director(id)
)
```

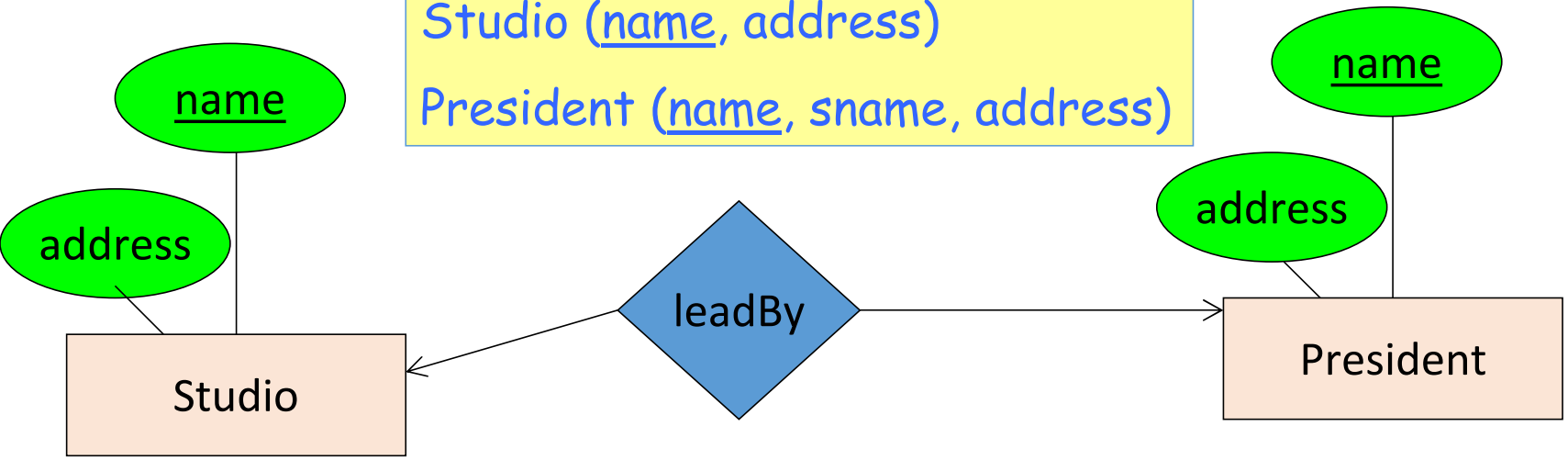
```
CREATE TABLE Director(
  id         INT,
  name    CHAR(40),
  PRIMARY KEY (id)
)
```

From E/R Diagrams to Relational Schema

Converting (1:1) Relationships to Relational Schema

Studio (name, address)

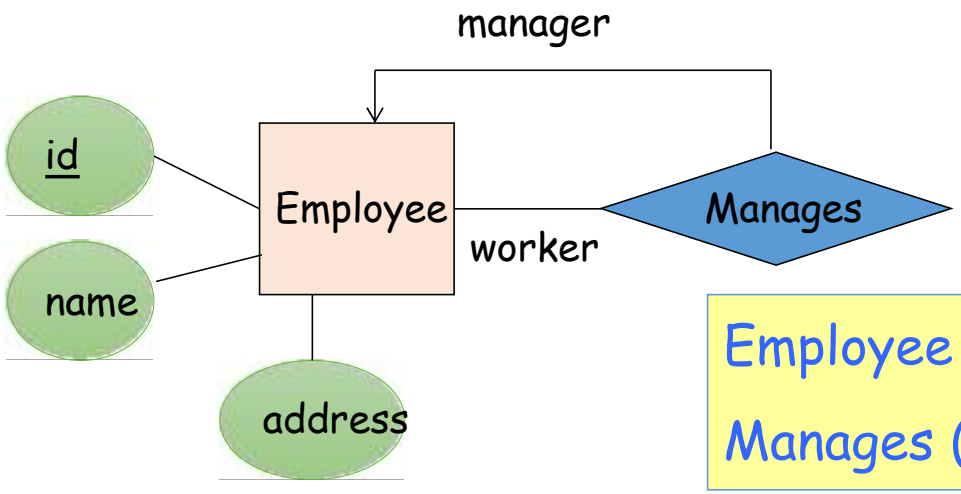
President (name, sname, address)




```
CREATE TABLE Studio(
  name   CHAR(40),
  address CHAR(40),
  PRIMARY KEY (name)
)
```

```
CREATE TABLE President(
  name        CHAR(40),
  sname       CHAR(40),
  address     VARCHAR(50),
  PRIMARY KEY (name),
   FOREIGN KEY (sname)
            REFERENCES Studio(name)
)
```

Translating Recursive Relationship Sets (First option)

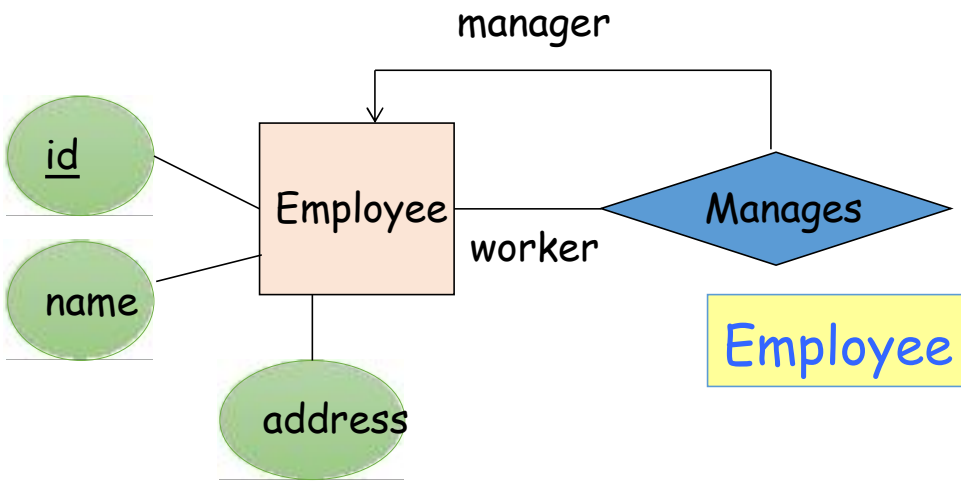


Employee (id, name, address)

Manages (wid, mid)

What are all the relations created for this diagram?

Translating Recursive Relationship Sets (second option)

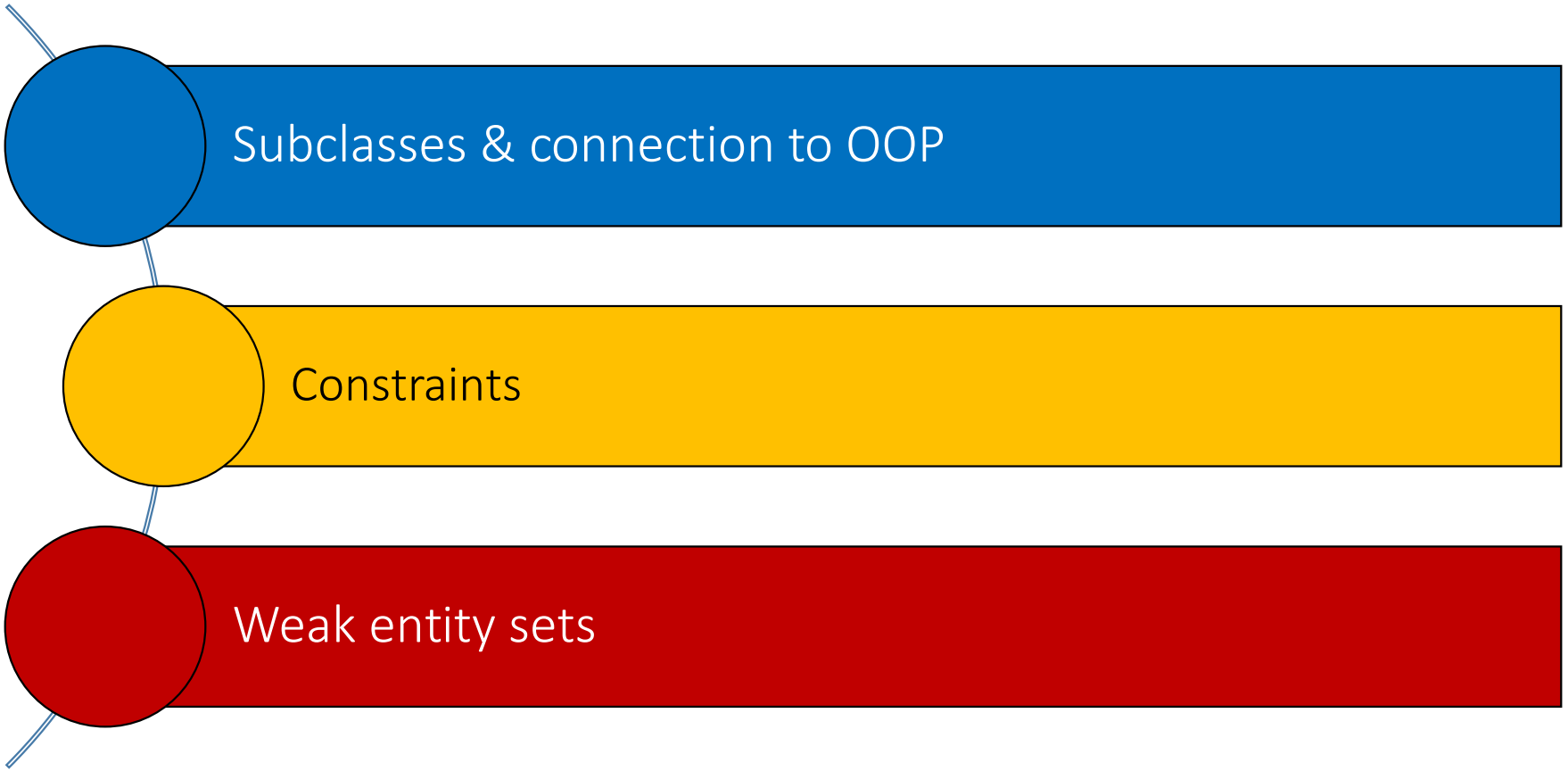


Employee (id, mid, name, address)

What are all the relations created for this diagram?

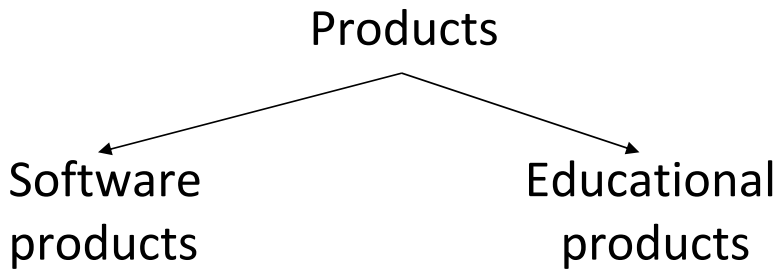
3. Advanced E/R Concepts

What you will learn about in this section



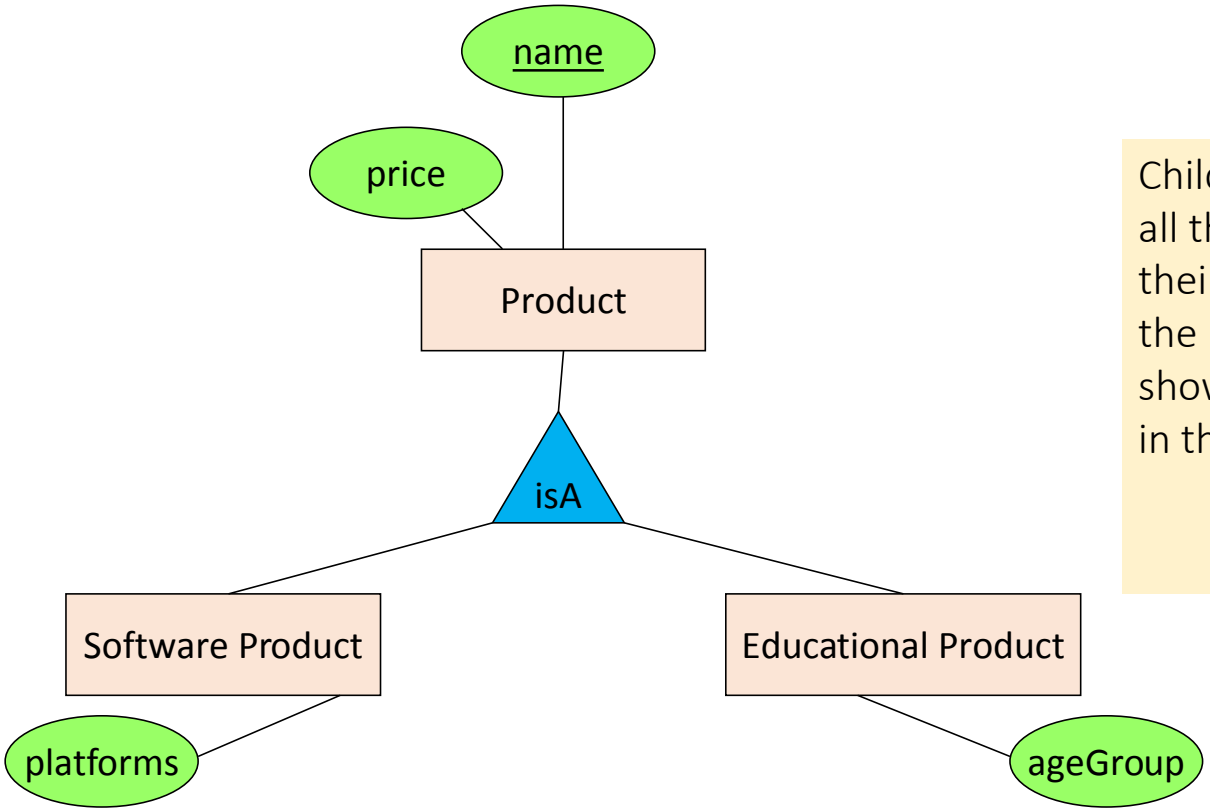
Modeling Subclasses

- Some objects in a class may be special, i.e. worthy of their own class
  - Define a new class?
    - *But what if we want to maintain connection to current class?*
  - Better: define a *subclass*
    - *Ex:*



We can define **subclasses** in E/R!

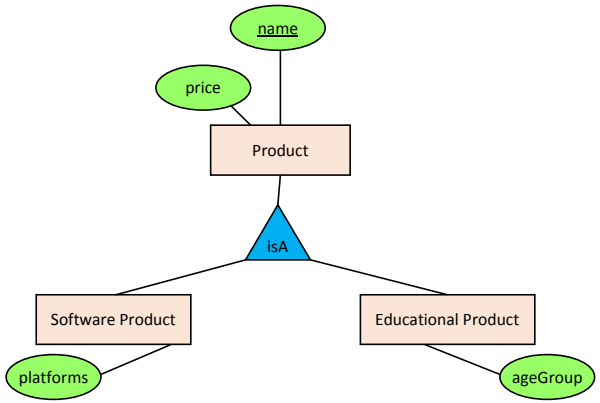
Modeling Subclasses



Child subclasses contain all the attributes of *all* of their parent classes **plus** the new attributes shown attached to them in the E/R diagram

Understanding Subclasses

- Think in terms of records; ex:



- Product

name
price

- SoftwareProduct

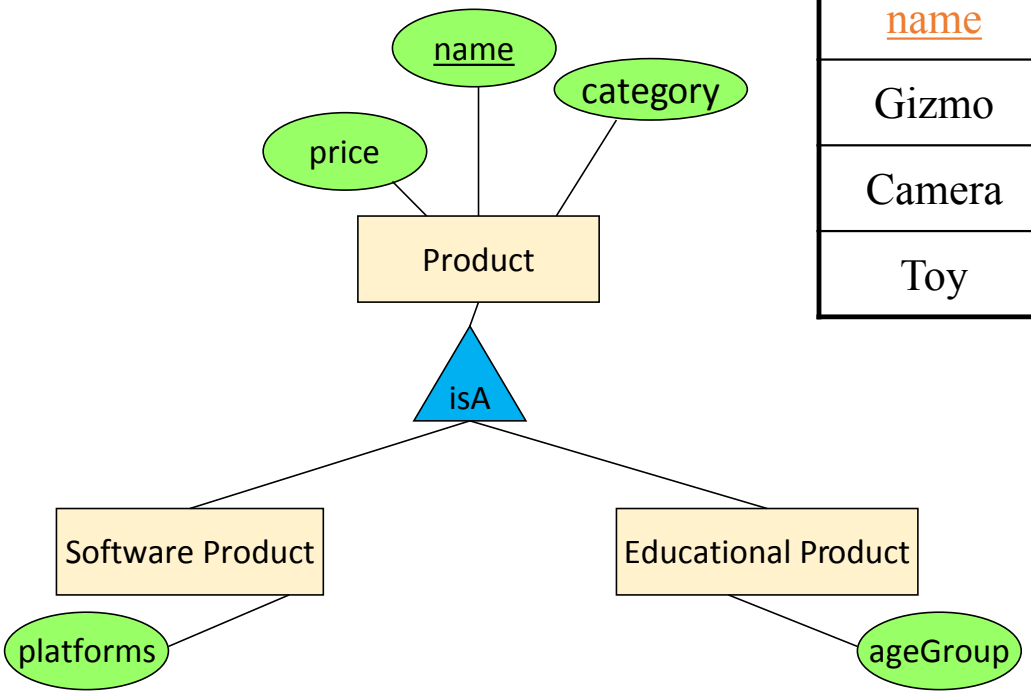
name
price
platforms

- EducationalProduct

name
price
ageGroup

Child subclasses contain all the attributes of *all* of their parent classes **plus** the new attributes shown attached to them in the E/R diagram

How to convert E/R to tables…

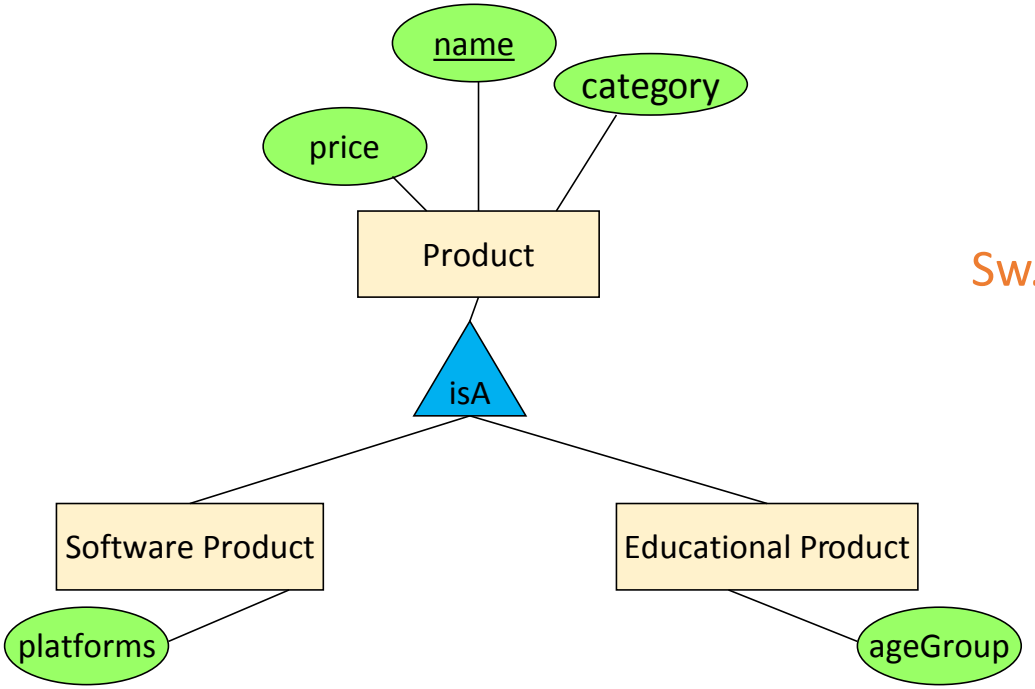


Product

name	price	category	platforms	ageGroup
Gizmo	99	gadget	unix	null
Camera	49	photo	null	null
Toy	39	gadget	null	retired

Many null values!!

Better option: one table for each subclass…



Product

name	price	category
Gizmo	99	gadget
Camera	49	photo
Toy	39	gadget

Sw.Product

name	platforms
Gizmo	unix

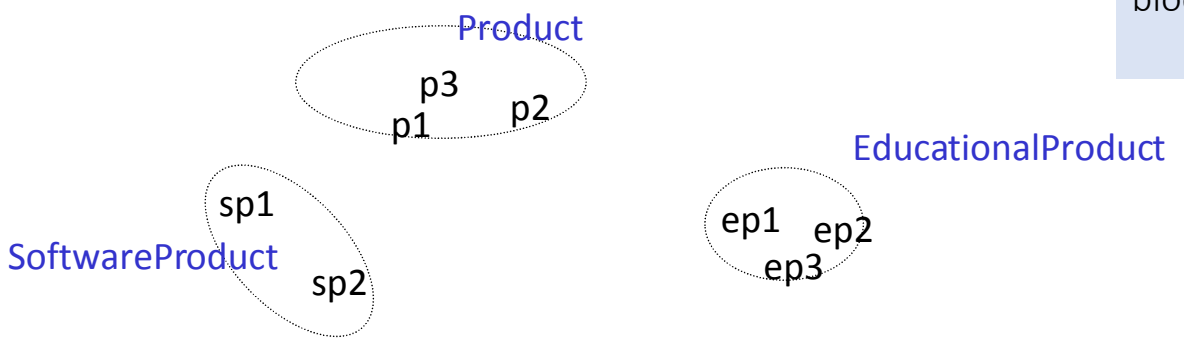
Ed.Product

name	ageGroup
Toy	retired

Difference between OOP and E/R inheritance

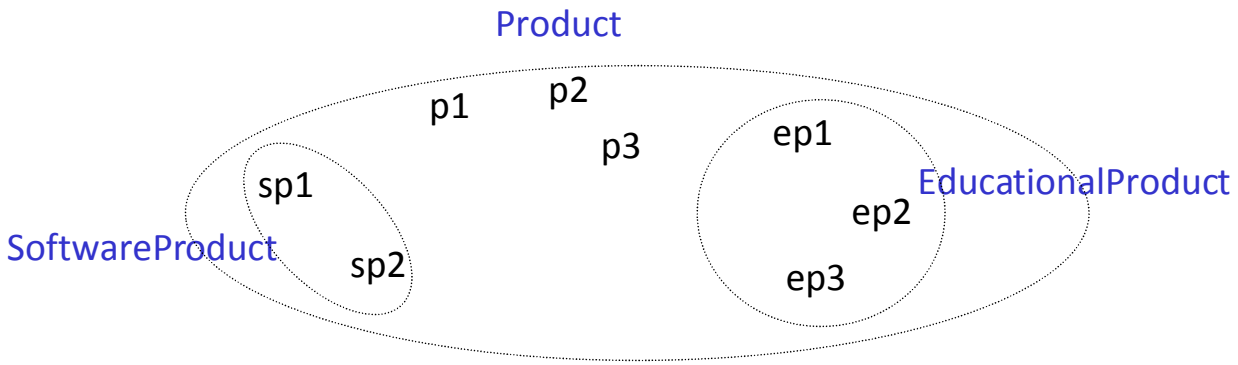
- OOP: Classes are disjoint (same for Java, C++)

OO = Object Oriented. E.g. classes as fundamental building block, etc…



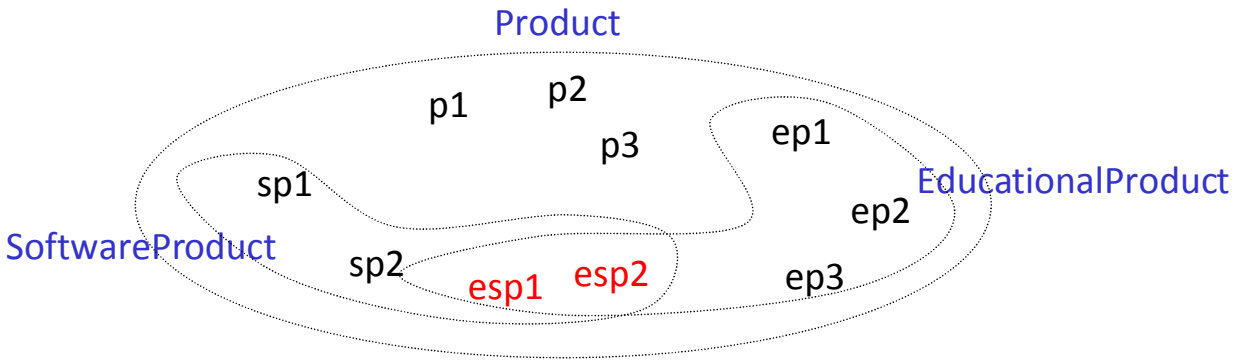
Difference between OOP and E/R inheritance

- E/R: entity sets overlap



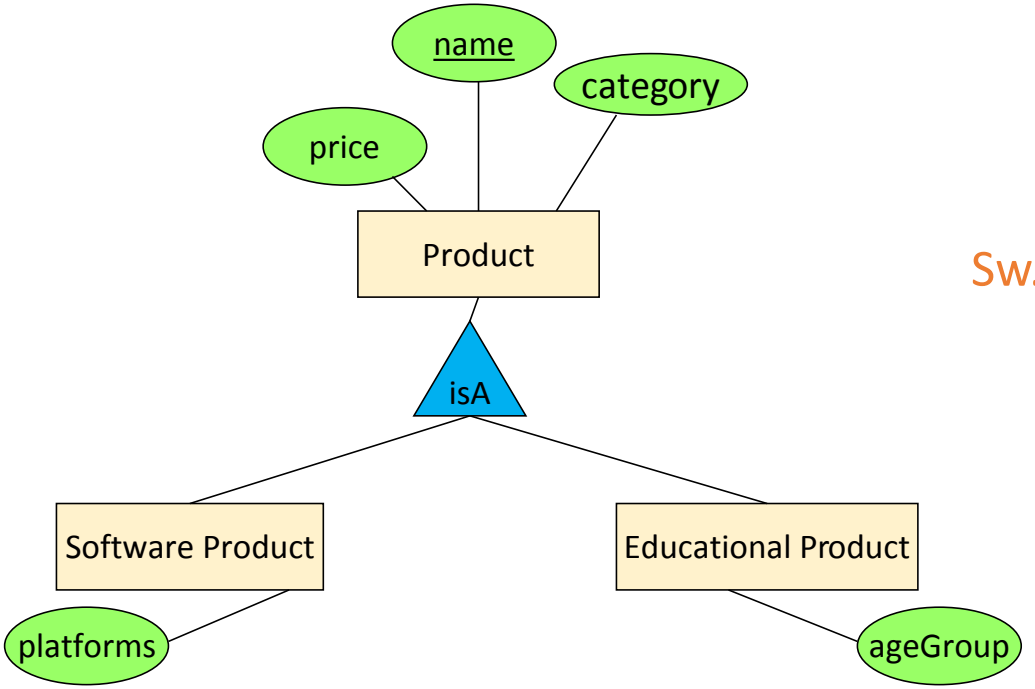
Difference between OO and E/R inheritance

We have three entity sets, but four different kinds of objects



No need for multiple inheritance in E/R

Suppose we have a product KIDPIX which is an educational software… Where do we keep the information?



Product

name	price	category
Gizmo	99	gadget
Camera	49	photo
Toy	39	gadget

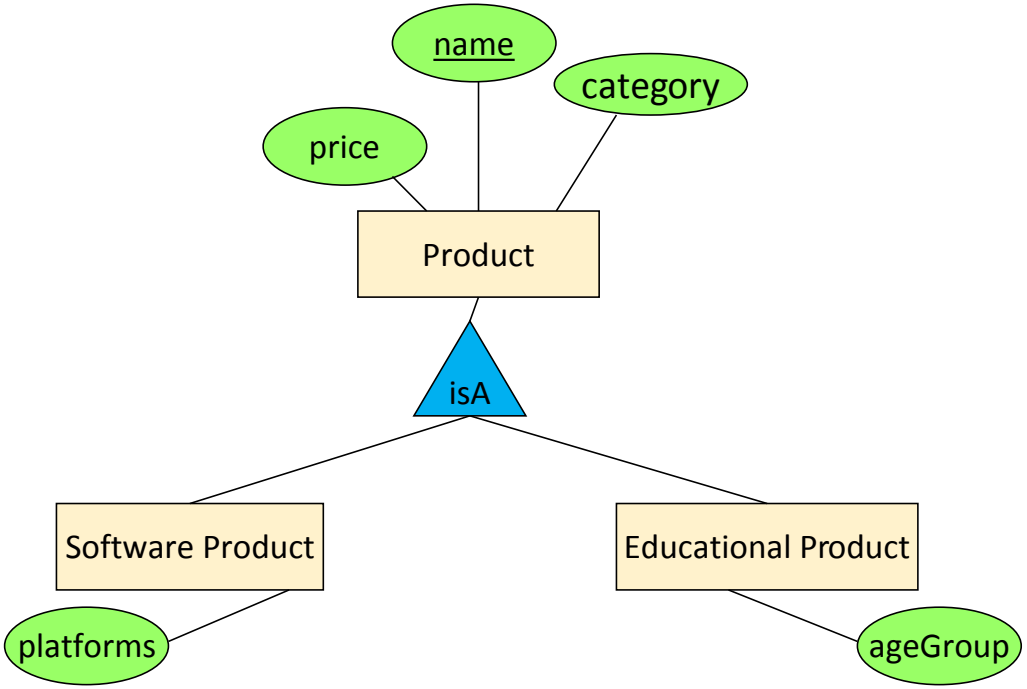
Sw.Product

name	platforms
Gizmo	unix

Ed.Product

name	ageGroup
Gizmo	todler
Toy	retired

Suppose we have a product KIDPIX which is an educational software… Where do we keep the information?



Product

name	price	category
Gizmo	99	gadget
Camera	49	photo
Toy	39	gadget
KidPix	100	gadget

name	platforms
Gizmo	unix
KidPix	Macintosh

Sw.Product

name	ageGroup
Toy	retired
KidPix	todler

Ed.Product

IsA Review

- If we declare ***A IsA B*** then every **A** is a **B**
- We use IsA to
  - Add descriptive attributes to a subclass
  - To identify entities that participate in a relationship
  - No table for IsA relationship
- **No need for multiple inheritance**

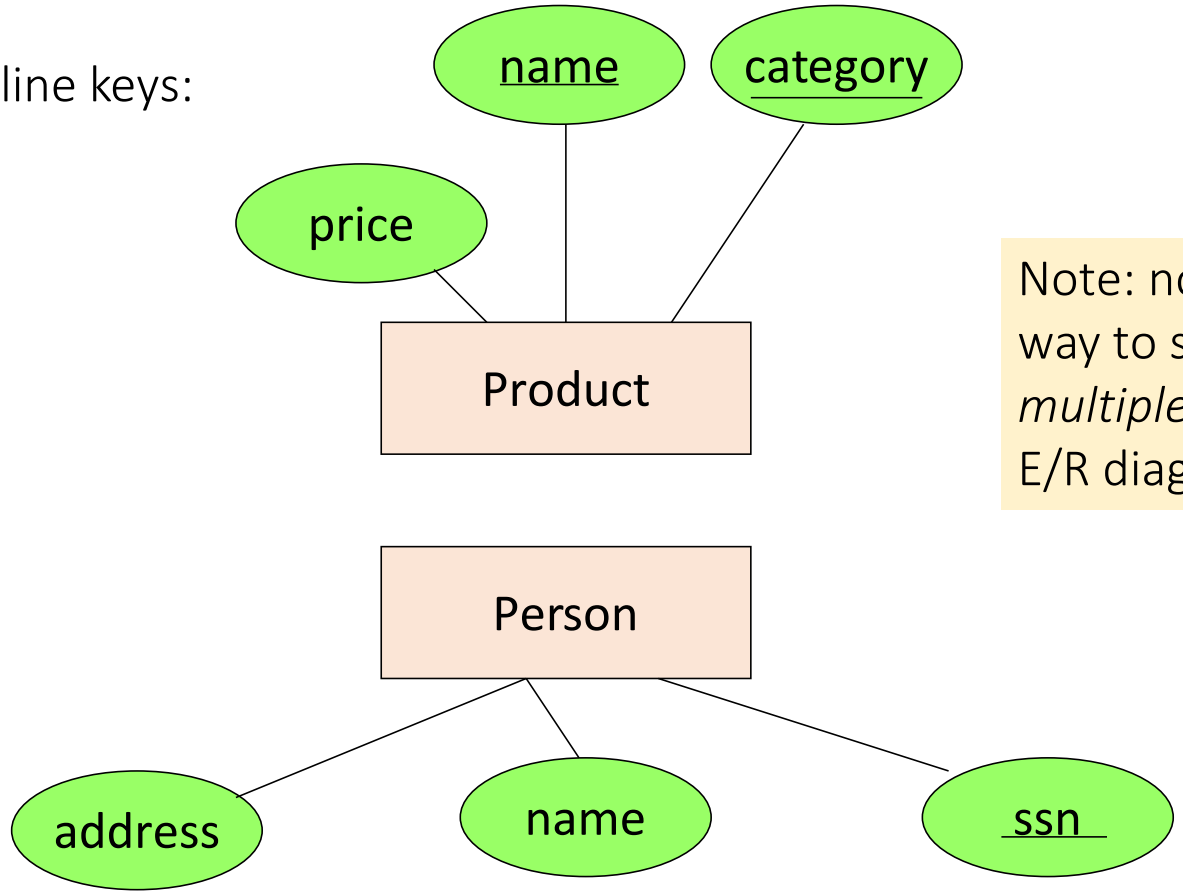
Constraints in E/R Diagrams

- Finding constraints is part of the E/R modeling process. Commonly used constraints are:
  - Keys: Implicit constraints on uniqueness of entities
    - *Ex: An SSN uniquely identifies a person*
  - Single-value constraints:
    - *Ex: a product can be made by only one company*
  - Referential integrity constraints: Referenced entities must exist
    - *Ex: if you work for a company, it must exist in the database*
  - Degree constraints:
    - *Ex: A movie may not have more than 10 actors*

Recall FOREIGN KEYs!

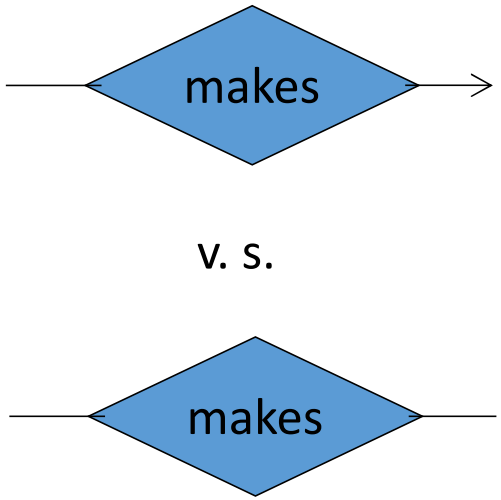
Keys in E/R Diagrams

Underline keys:

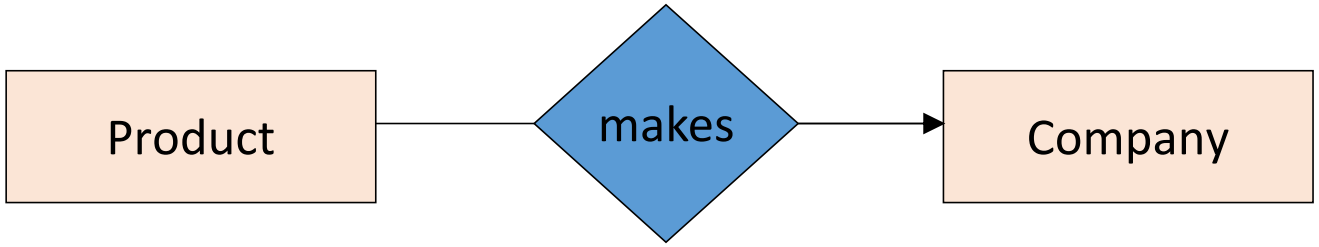


Note: no formal way to specify *multiple* keys in E/R diagrams…

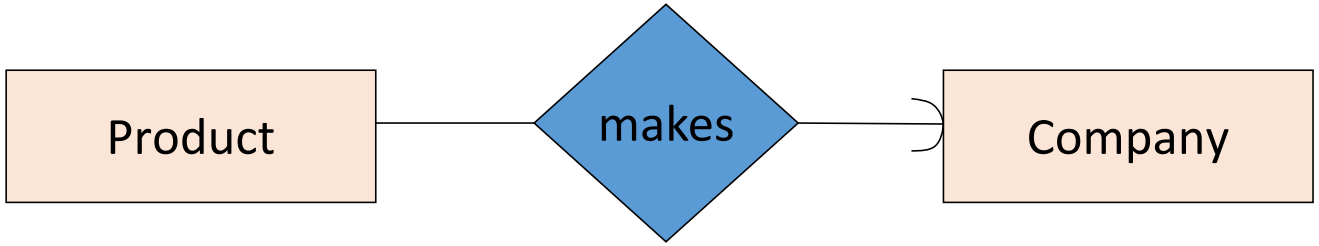
Single Value Constraints



Referential Integrity Constraints

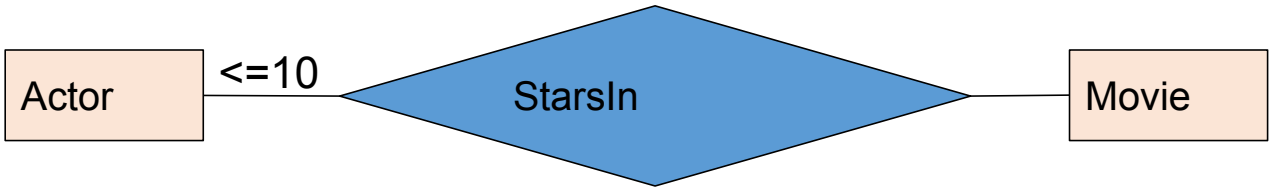


Each product made by at most one company.
Some products made by no company?



Each product is made by *exactly* one company.

Degree Constraints



- We can attach a bounding number to edges to indicate limits on the number of entities that can be connected to a single entity via a relationship set
- In the example above, a movie has at most 10 stars
- Note: a regular arrow is the constraint <=1
- Note: a rounded arrow is the constraint =1

Weak Entity Sets

The entity set that can not exist without another entity set

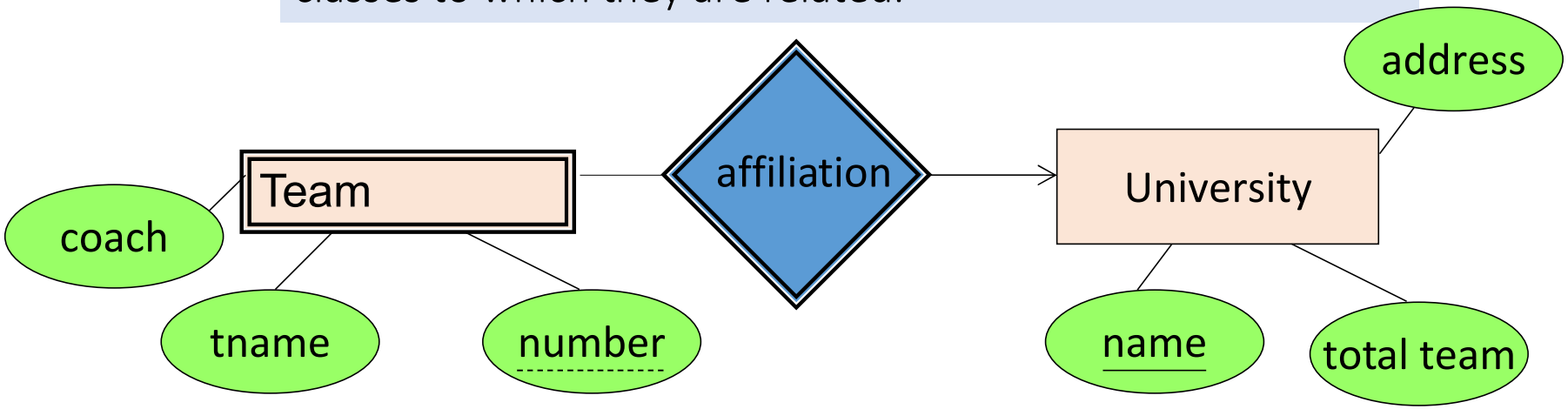
a ROOM can only exist in a BUILDING

a TIRE is considered as a strong entity because it can exist without being attached to a CAR

Question is strong entity type and answer is weak. Question is always there, but an answer requires a question to exist.

Weak Entity Sets

Entity sets are *weak* when their key comes from other classes to which they are related.



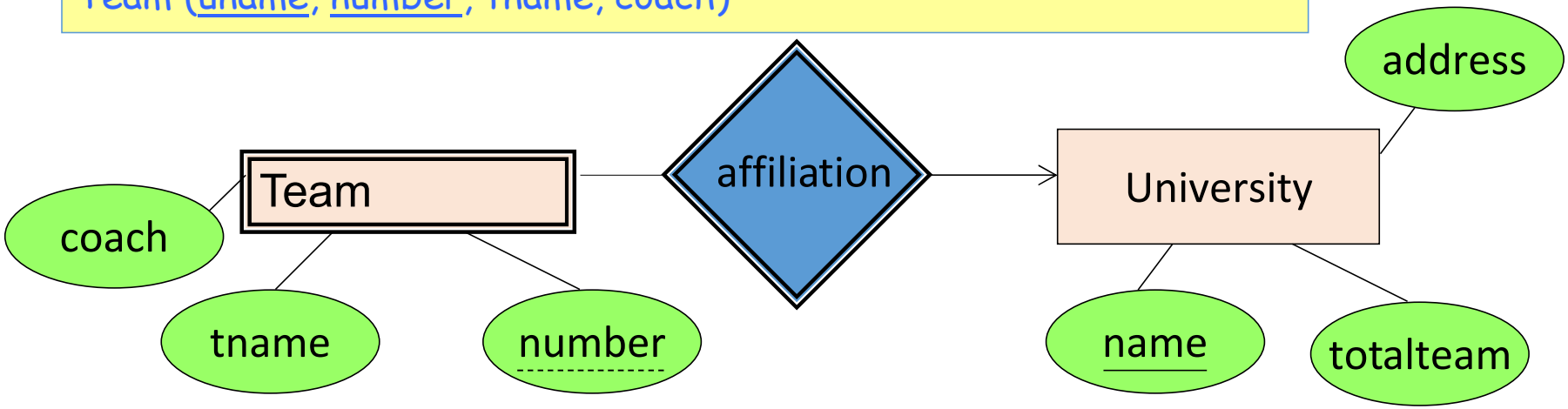
"Programming contest team" v. "***BUET Programming contest team***" *(E.g., Dhaka university has a programming contest team too)*

Weak Entity Sets

Only 2 tables are required, no table for the relationship is required

University (name, totalTeam, address)

Team (uname, number, tname, coach)



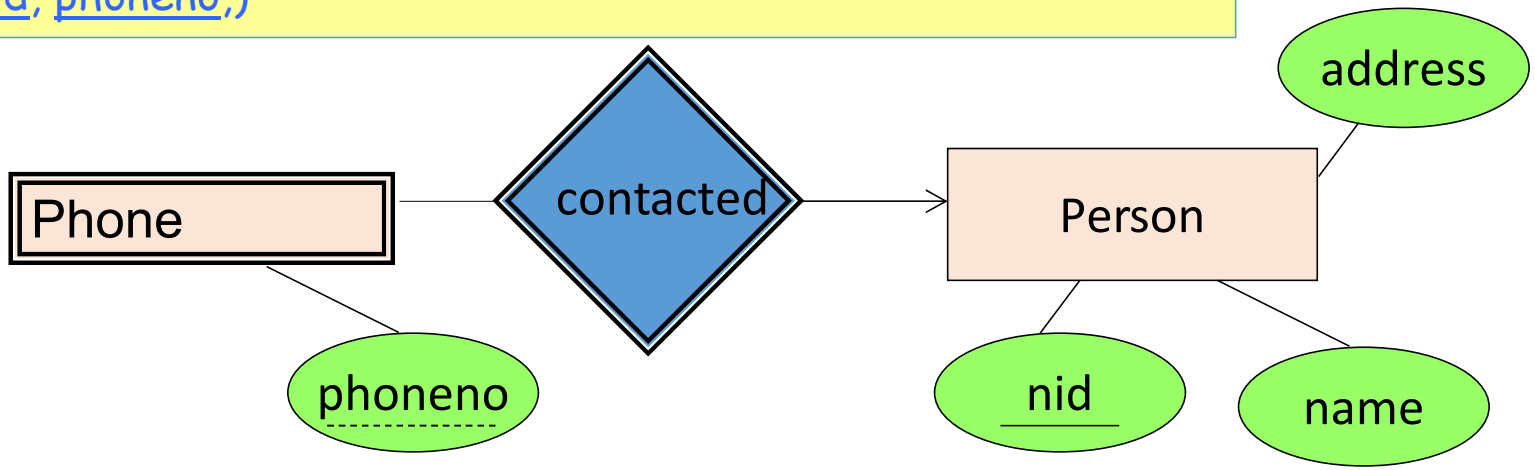
- number is a *partial key*. (denote with dashed underline).
- University is called the *identifying owner*.

Weak Entity Sets

Only 2 tables are required, no table for the relationship is required

Person (nid, name, address)

Phone (nid, phoneno,)



- Phone no is a *partial key*. (denote with dashed underline).
- Person is called the *identifying owner*.

E/R Summary

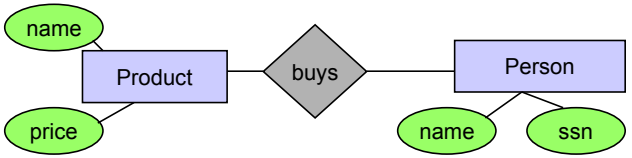
- E/R diagrams are a visual syntax that allows technical and non-technical people to talk
  - For conceptual design

- Basic constructs: **entity**, **relationship**, and **attributes**

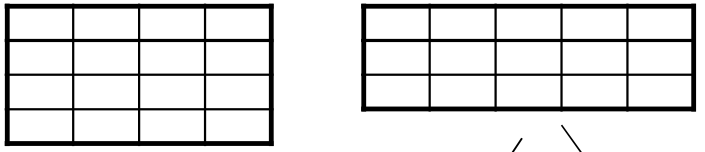
- A good design is faithful to the constraints of the application

The bigger picture of what we have learned

Conceptual Model:



Relational Model (in 1NF) plus FD's



Normalization:
Eliminates ***anomalies***